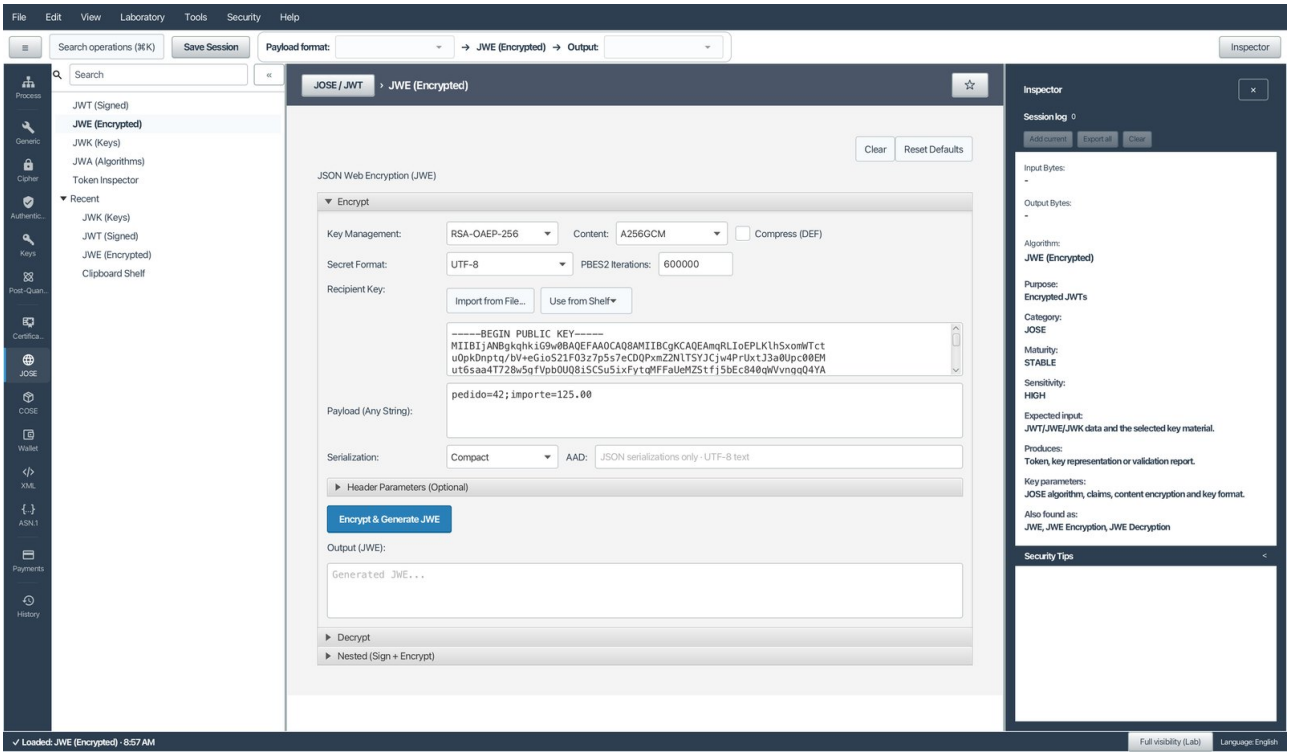
JOSE a fondo — JWE y JWK con todas las opciones

Este tutorial amplía JWT, JWS, JWE y JWK. Aquel explica el modelo mental y los casos básicos; este recorre **cada control** de la sección JOSE, con el foco en cifrado (JWE) y en gestión de claves (JWK/JWKS). JWS/JWT, el inspector y el registro JWA se cubren como referencia al final.

> Laboratorio educativo. Todas las claves y secretos de este documento son públicos: no los uses fuera de las pruebas.

Las etiquetas se citan en inglés, igual que en el tutorial 09. Con la interfaz en español verás el equivalente traducido (por ejemplo **Key Management** → *Gestión de claves*, **Secret Format** → *Formato del secreto*).



Mapa de la sección

Módulo del explorador	Paneles	Para qué
JWT (Signed)	Generate JWT · Validate & Decode JWT · JWS Detached	Firmar, validar firma y claims, payload externo
JWE (Encrypted)	Encrypt · Decrypt · Nested (Sign + Encrypt)	Cifrar, descifrar, firmar y después cifrar
JWK (Keys)	JWK Tools · JWK Set (JWKS)	Convertir claves, thumbprint, metadatos, conjuntos y rotación
JWA (Algorithms)	Tabla	Registro de algoritmos disponibles

Módulo del explorador	Paneles	Para qué
Token Inspector	Analyze Token	Desglosar cualquier JWS/JWE sin clave

Arriba a la derecha, **Clear** vacía todos los campos del módulo y **Reset Defaults** además devuelve los selectores a sus valores seguros.

Material de laboratorio

Se usa a lo largo de todo el tutorial.

Nombre	Valor
Payload	`pedido=42;importe=125.00` (24 bytes)
Clave pública RSA	clave-publica-laboratorio.pem
Par RSA con privada	clave-privada-laboratorio-comparacion.pem y su pública
Secreto de 256 bits (Hex)	`000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f`
El mismo secreto (Base64)	`AAECAwQFBgcICQoLDA0ODxAREhMUFRYXGBkaGxwdHh8=`
Secreto de 128 bits (Hex)	`000102030405060708090a0b0c0d0e0f`
Contraseña PBES2	`correct horse battery staple`

Para cifrar basta la clave pública; para el viaje de ida y vuelta completo usa el par de comparación, que incluye la privada.

Parte 1 — Cómo se introducen las claves

Todos los campos de clave del módulo aceptan los mismos formatos. Entenderlos una vez evita la mayoría de los errores.

Claves asimétricas (RSA, EC, OKP)

Formato	Ejemplo de cabecera	Notas
PEM SubjectPublicKeyInfo	`-----BEGIN PUBLIC KEY-----`	Pública
PEM PKCS#1	`-----BEGIN RSA PRIVATE KEY-----`	RSA tradicional
PEM PKCS#8	`-----BEGIN PRIVATE KEY-----`	RSA, EC, Ed25519/Ed448, X25519/X448
PEM SEC1	`-----BEGIN EC PRIVATE KEY-----`	Debe nombrar su curva
Certificado X.509	`-----BEGIN CERTIFICATE-----`	Se extrae la pública
DER en Base64 sin cabeceras	`MIIBIjANBg…`	PKCS#8 o SubjectPublicKeyInfo
JWK	`{"kty":"RSA",…}`	Pública o privada
JWKS de **una** clave	`{"keys":[{…}]}`	Con más de una clave: *The JWKS holds N keys; paste the single JWK to use*

Reglas que conviene recordar:

- **Una privada sirve donde se pide la pública.** La pública se deriva, así que puedes cifrar o verificar con el mismo material con el que descifras o firmas.
- **Los PEM cifrados con contraseña no se admiten.** Descífralos antes en Claves.
- **Import from File…** carga el archivo tal cual; **Use from Shelf** inserta una clave guardada en la repisa.

Secretos compartidos y contraseñas

Los algoritmos simétricos (`A*KW`, `A*GCMKW`, `dir`, `PBES2-*`, `HS*`) leen el campo de clave como texto y lo convierten a bytes según **Secret Format**:

Secret Format	Cómo se interpreta	Cuándo usarlo
`UTF-8` (por defecto)	Los caracteres tal cual	Contraseñas PBES2 y secretos tecleados
`Hex`	Pares hexadecimales; admite espacios y prefijo `0x`	Claves AES exactas
`Base64 / Base64URL`	Cualquiera de las dos variantes, con o sin relleno	Valor `k` de una JWK `oct`

Dos trampas frecuentes:

1. **El formato cambia la longitud.** `0123456789abcdef` en `UTF-8` son 16 bytes (válido para A128KW); en `Hex` son 8 bytes (inválido).
2. **Una JWK `oct` no es un secreto.** Pegar `{"kty":"oct","k":"…"}` en un campo de secreto no extrae `k`: con AES-KW falla por longitud y con PBES2 se usa *todo el JSON* como contraseña. Convierte primero con **JWK → Secret** (Parte 3) y pega el valor Hex o Base64.

Cada panel tiene su propio selector: el de **Encrypt** no afecta a **Decrypt**. Si cifras en `Hex` y descifras en `UTF-8`, fallará.

Parte 2 — JWE (Encrypted)

Un JWE combina dos decisiones independientes:

- **Key Management (`alg`)**: cómo llega al destinatario la clave de contenido (CEK).
- **Content (`enc`)**: con qué cifrado autenticado se protege el payload usando esa CEK.

2.1 Panel Encrypt, campo a campo

Control	Qué hace
Key Management	Algoritmo `alg`. Diecinueve opciones (tabla 2.2). Por defecto la primera de la lista, `RSA1_5`, marcada como insegura: **cámbiala**
Content	Algoritmo `enc`. Por defecto `A256GCM`
Compress (DEF)	Añade `zip: DEF` y comprime el payload con DEFLATE antes de cifrar
Secret Format	Cómo leer secretos y contraseñas. Se ignora con claves RSA/EC/OKP
PBES2 Iterations	Iteraciones PBKDF2. Por defecto `600000`; mínimo aceptado `1000`. Solo afecta a `PBES2-*`
Recipient Key	Clave del destinatario: pública RSA/EC/OKP, JWKS con varios destinatarios, secreto o contraseña
Payload (Any String)	Texto a cifrar. No tiene que ser JSON
Serialization	`Compact`, `Flattened JSON` o `General JSON` (2.5)
AAD	Datos autenticados adicionales, en texto UTF-8. Solo en serializaciones JSON
Header Parameters (Optional)	`kid`, `typ`, `cty`, `apu`, `apv` y parámetros propios (2.4)
Encrypt & Generate JWE	Ejecuta y escribe el resultado en **Output (JWE)**

Junto a **Key Management** aparece un aviso cuando eliges un algoritmo débil. En el desplegable, `RSA1_5` y `RSA-OAEP` llevan el sufijo *Aviso: inseguro*.

2.2 Algoritmos de gestión de clave (`alg`)

`alg`	Recipient Key	Segmento `encrypted_key`	Vista previa de CEK	Observaciones
`RSA1_5`	Pública RSA	256 bytes con RSA-2048	Sí	**Inseguro**: oráculo de relleno (Bleichenbacher). Solo para interoperar con sistemas heredados
`RSA-OAEP`	Pública RSA	Igual	Sí	OAEP con SHA-1; desaconsejado en diseños nuevos
`RSA-OAEP-256` / `-384` / `-512`	Pública RSA	Igual	Sí	Opción RSA recomendada
`ECDH-ES`	Pública EC (P-256/384/521) u OKP (X25519/X448)	**Vacío**	Sí con EC, no con OKP	Acuerdo directo: la CEK se deriva, no se transporta
`ECDH-ES+A128KW` / `+A192KW` / `+A256KW`	Igual	CEK envuelta	Sí con EC, no con OKP	Acuerdo + AES Key Wrap
`A128KW` / `A192KW` / `A256KW`	Secreto de 16 / 24 / 32 bytes	CEK envuelta	Sí	El tamaño debe ser exacto

alg	Recipient Key	Segmento encrypted_key	Vista previa de CEK	Observaciones
A128GCMKW / A192GCMKW / A256GCMKW	Secreto de 16 / 24 / 32 bytes	CEK envuelta	No	Añade iv y tag a la cabecera
PBES2-HS256+A128KW / -HS384+A192KW / -HS512+A256KW	Contraseña	CEK envuelta	Sí	Añade p2s (sal de 16 bytes) y p2c (iteraciones)
dir	Secreto del tamaño que exige enc	**Vacío**	No, deliberadamente	El secreto *es* la CEK

Si el secreto no mide lo que pide el algoritmo, el mensaje lo dice con precisión:

```
A128KW needs a 128-bit key (16 bytes); the supplied secret has 32 bytes. Check the key format (U
TF-8 / Hex / Base64).
```

Con una clave de tipo equivocado (por ejemplo EC en RSA-OAEP-256): *The supplied key is not an RSA key.*

2.3 Algoritmos de contenido (enc)

enc	CEK	IV	Tag	Tamaño de secreto con dir
A128GCM	16 bytes	12 bytes	16 bytes	16 bytes
A192GCM	24 bytes	12 bytes	16 bytes	24 bytes
A256GCM	32 bytes	12 bytes	16 bytes	32 bytes
A128CBC-HS256	32 bytes	16 bytes	16 bytes	**32 bytes**
A192CBC-HS384	48 bytes	16 bytes	24 bytes	**48 bytes**
A256CBC-HS512	64 bytes	16 bytes	32 bytes	**64 bytes**

Las variantes CBC-HS usan una CEK doble: la primera mitad autentica (HMAC) y la segunda cifra (AES-CBC). Por eso dir con A128CBC-HS256 pide 32 bytes y no 16. Con el resto de algoritmos alg la CEK se genera sola y no tienes que preocuparte del tamaño.

2.4 Parámetros de cabecera

Despliega **Header Parameters (Optional)**. Los campos vacíos no se emiten.

Campo	Cabecera	Uso
Key ID (kid)	kid	Selector de la clave del destinatario
Type (typ)	typ	Tipo del objeto completo, p. ej. JWT
Content Type (cty)	cty	Tipo del payload. JWT si lo cifrado es un token firmado
Party U Info (apu)	apu	Identidad del emisor en el acuerdo ECDH-ES. Escribe texto; se envía en Base64URL
Party V Info (apv)	apv	Identidad del receptor, igual que apu
Custom Parameters	Cualquiera	Objeto JSON que se fusiona con la cabecera

Detalles de comportamiento:

- apu y apv entran en la derivación Concat KDF de ECDH-ES*: si cambian, cambia la clave. Con otros algoritmos se escriben en la cabecera pero no afectan a la criptografía.

- **Custom Parameters** no puede fijar `alg`, `enc`, `zip`, `epk`, `iv`, `tag`, `p2s` ni `p2c`; los eligen los campos de algoritmo. El intento se rechaza: *Custom header parameters cannot set 'enc'; it is chosen by the algorithm fields.*
- Sí admite `crit`, `x5t#S256`, `iss`, `aud` y cualquier nombre propio. Si repite un nombre que ya rellenaste arriba (`kid`, `typ`, `cty`), gana el JSON.
- En serialización compacta toda la cabecera es **protegida**: va autenticada por el tag.

Ejemplo de parámetros propios:

```
{"x5t#S256": "kTnWcRPLLePOgTMIlsajJGeJYiO_JpQJyStC7q_GMM8", "iss": "cryptocarver-lab"}
```

2.5 Serializaciones y AAD

Serialization	Forma	AAD	Destinatarios
`Compact`	`protected.encrypted_key.iv.ciphertext.tag`	No	Uno
`Flattened JSON`	Objeto con `protected`, `encrypted_key`, `iv`, `ciphertext`, `tag`	Sí	Uno
`General JSON`	Igual, con un array `recipients`	Sí	Uno o varios

El campo **AAD** es texto UTF-8; en el JSON aparece como miembro `aad` en Base64URL (`ctx` → `Y3R4`). El tag autentica `protected + "." + aad`, de modo que alterar el AAD invalida el descifrado aunque el ciphertext esté intacto. Con `Compact` y AAD relleno la operación se rechaza: *AAD needs a JSON serialization; compact JWE has no 'aad' member.*

Dónde viajan los parámetros que añade el propio algoritmo (`epk`, `p2s`/`p2c`, `iv`/`tag` de GCMKW) depende del AAD:

- **Sin AAD**, con un destinatario: toda la cabecera va en `protected`, igual que en compacto.
- **Con AAD**: van en un miembro `unprotected`, fuera de `protected`.
- **Con varios destinatarios**: van en la cabecera `header` de cada uno.

2.6 Varios destinatarios

`General JSON` permite cifrar un mismo contenido para varios destinatarios: una sola CEK, envuelta una vez por cada uno.

1. Prepara un JWKS con **dos o más** claves. Cada una debe declarar su `alg`; el `kid` es opcional pero recomendable.
2. Pégalo en **Recipient Key**.
3. Elige **Serialization = General JSON**.
4. Deja vacíos **kid**, **apu** y **apv**: son por destinatario y se toman de cada JWK.
5. Cifra.

En este modo **Key Management** no se usa: manda el `alg` de cada clave. Forma del resultado, abreviada:

```
{
  "protected": "eyJlbmMiOiJBMjU2R0NNIn0",
  "aad": "Y3R4",
  "recipients": [
    {"header": {"alg": "RSA-OAEP-256", "kid": "rsa-1"}, "encrypted_key": "Jlxs…"},
    {"header": {"alg": "ECDH-ES+A256KW", "kid": "ec-1", "epk": {"kty": "EC", "crv": "P-256", "x"
: "…", "y": "…"}}, "encrypted_key": "5dFj…"}
  ],
  "iv": "Fg-Ucu7RdNphr33H",
  "ciphertext": "Ixk5Dw",
  "tag": "4YN1XJW8tLcsIre_gEPelA"
}
```

La cabecera protegida solo lleva `enc`; el `alg` vive en la cabecera de cada destinatario.

Situación	Resultado
Alguna JWK sin `alg`	*Each recipient JWK needs an 'alg' member (key rsa-1).*
`kid`, `apu` o `apv` rellenos	*kid, apu and apv are per recipient; with several recipients they come from each JWK.*
JWKS de varias claves con `Compact` o `Flattened JSON`	*The JWKS holds 2 keys; paste the single JWK to use.*
Destinatarios RSA, EC (P-256/384/521), OKP (X25519/X448) y `oct` mezclados	Funciona
Algún destinatario con `alg: ECDH-ES` a secas	Se rechaza: deriva su propia CEK y no puede compartirla; usa `ECDH-ES+A*KW`
Un destinatario `oct` con `alg: dir`	Funciona: su clave es la CEK y los demás la reciben envuelta. Solo puede haber uno

Para construir el JWKS puedes usar **JWK Set (JWKS)** (Parte 3): las claves que genera ya llevan `alg` y `kid`.

2.7 Panel Decrypt

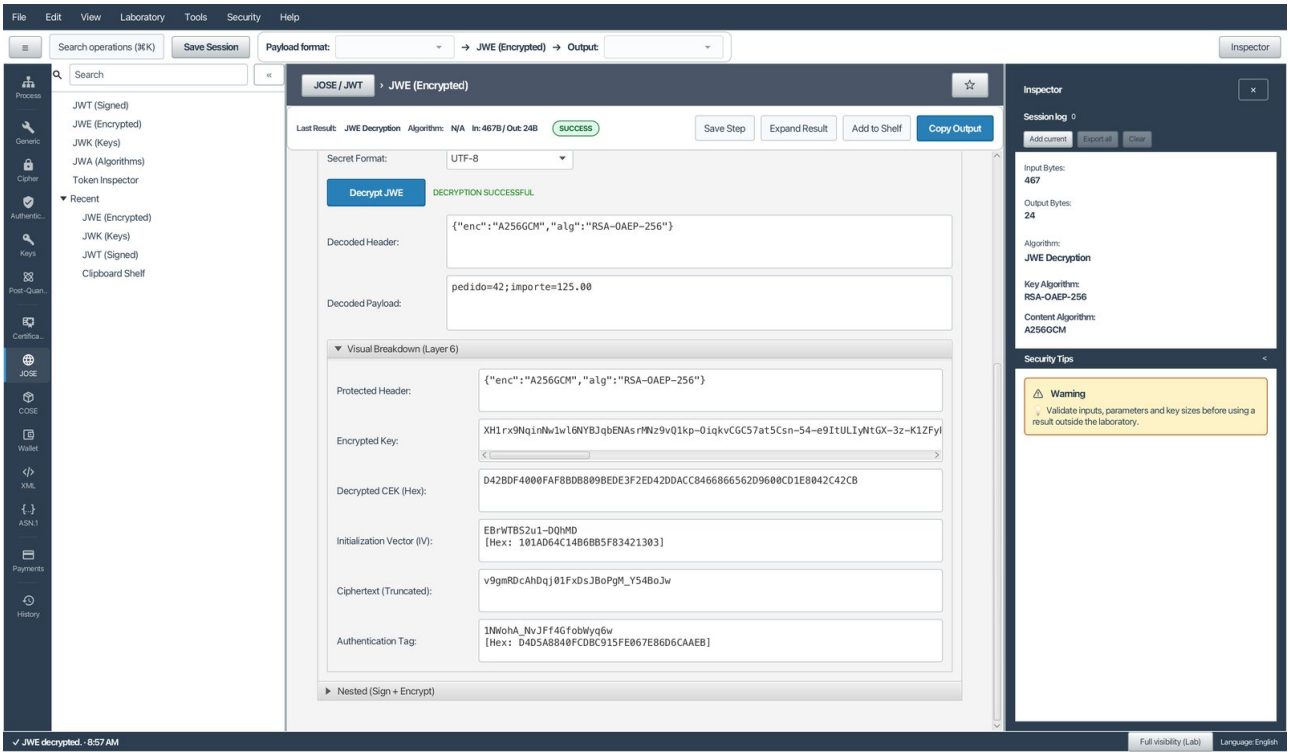
Control	Qué hace
JWE Token	Compacto o JSON. Se detecta solo: si empieza por `{` se trata como JSON
Private Key / Secret	Privada RSA/EC/OKP, secreto o contraseña
Secret Format	Independiente del de Encrypt. Debe coincidir con el usado al cifrar
Decrypt JWE	Ejecuta. El estado muestra *DESCIFRADO CORRECTO* o *DESCIFRADO FALLIDO*
Decoded Header / **Decoded Payload**	Cabecera efectiva y texto recuperado
Visual Breakdown (Layer 6)	Las piezas del token por separado

No hay que elegir algoritmo: se lee de la cabecera. Con JSON de varios destinatarios se prueba cada uno con tu clave, sin necesidad de que el `kid` coincida, y el estado indica cuál encajó: *DESCIFRADO CORRECTO (recipient 2/2)*.

Desglose visual

Campo	Contenido
Protected Header	JSON de la cabecera
Encrypted Key	CEK envuelta en Base64URL. Vacío con `dir` y `ECDH-ES`
Decrypted CEK (Hex)	La CEK recuperada, cuando está disponible

Campo	Contenido
Initialization Vector (IV)	Base64URL y Hex
Ciphertext	Base64URL
Authentication Tag	Base64URL y Hex



Visual Breakdown: cabecera, clave cifrada, CEK en claro, IV, ciphertext y tag

La CEK en claro permite comprobar a mano el cifrado de contenido en Cifrado: con `A256GCM`, la clave es la CEK, el nonce es el IV y el AAD son los bytes ASCII del primer segmento del token. No siempre se muestra:

Caso	Lo que verás
`RSA*`, `A*KW`, `PBES2-*`, `ECDH-ES*` con clave EC	CEK en hexadecimal
`dir`	*Direct encryption: the CEK is the supplied direct key and is not displayed automatically.*
`A*GCMKW`	*Manual CEK preview is not available for A256GCMKW.*
`ECDH-ES*` con X25519/X448	*Manual CEK preview error: The supplied private key is incompatible…* (el descifrado sí funciona)
Serialización JSON	*Manual CEK preview is only available for compact serialization.*

La CEK se muestra solo en pantalla: no pasa al historial, a los informes ni a los registros. El payload descifrado se clasifica como secreto en Historial.

Errores de descifrado

Cualquier fallo criptográfico produce el mismo mensaje, sin distinguir la causa:

```
JWE authentication failed for RSA-OAEP-256: the supplied key may be incorrect/incompatible or th
e JWE header/ciphertext may be corrupt.
```

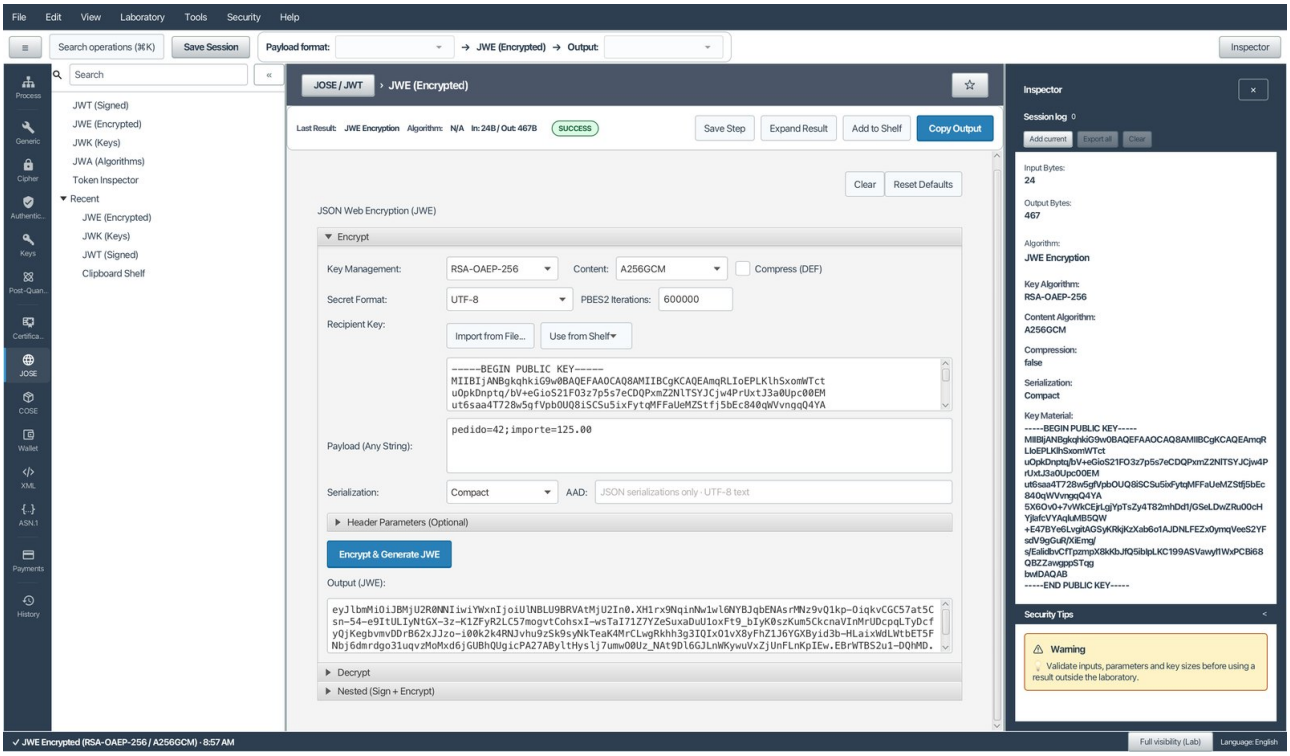
Es intencionado: diferenciar "clave incorrecta" de "tag incorrecto" ante quien envía el token facilita oráculos de descifrado.

2.8 Laboratorios JWE

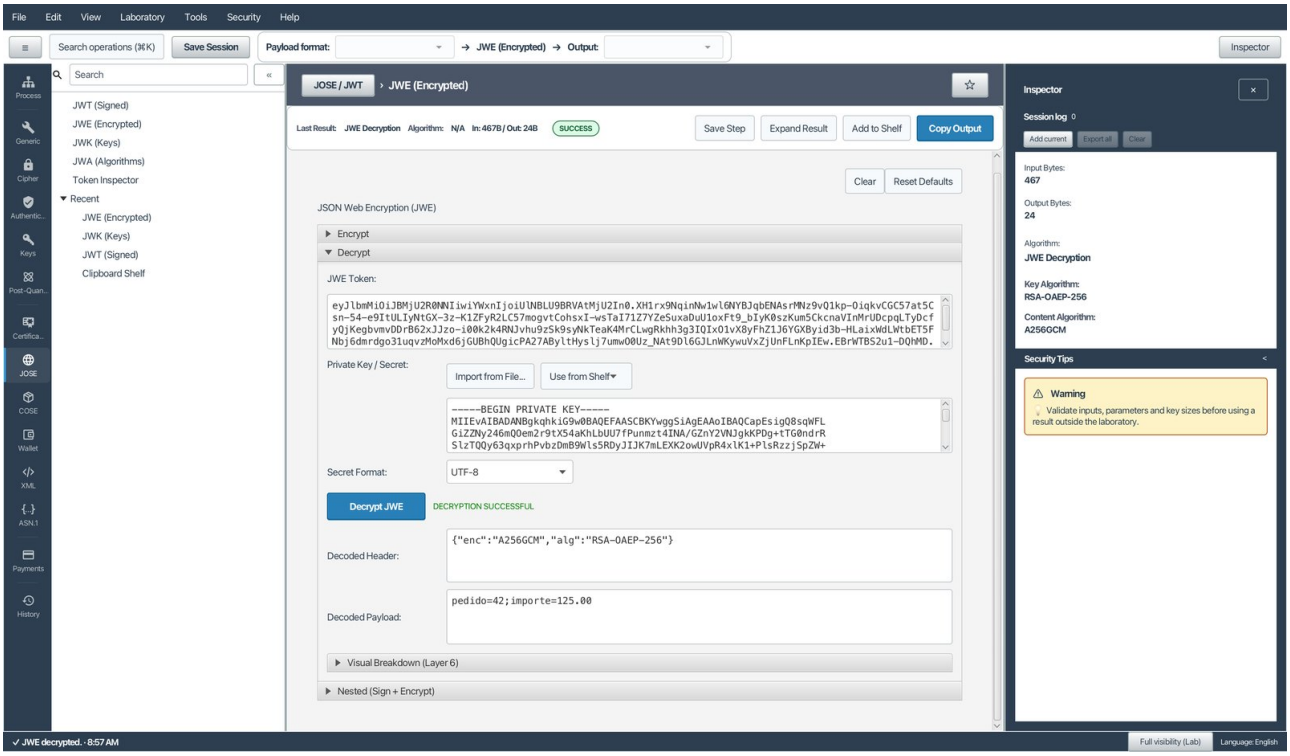
Todos usan el payload `pedido=42;importe=125.00`. Como IV, CEK y claves efímeras son aleatorios, el token cambia en cada ejecución: lo reproducible son los parámetros, la estructura, el tamaño aproximado y que el descifrado devuelva exactamente la entrada.

Laboratorio A — RSA-OAEP-256 + A256GCM

1. **Encrypt**: `Key Management = RSA-OAEP-256`, `Content = A256GCM`, `Serialization = Compact`.
2. **Recipient Key**: clave-publica-laboratorio-comparacion.pem.
3. Cifra. Resultado: cinco segmentos, 467 caracteres.
4. **Decrypt**: pega el token y clave-privada-laboratorio-comparacion.pem.
5. Comprueba el payload y que **Decrypted CEK (Hex)** tiene 64 dígitos (32 bytes).



Encrypt & Generate JWE: token compacto de cinco segmentos

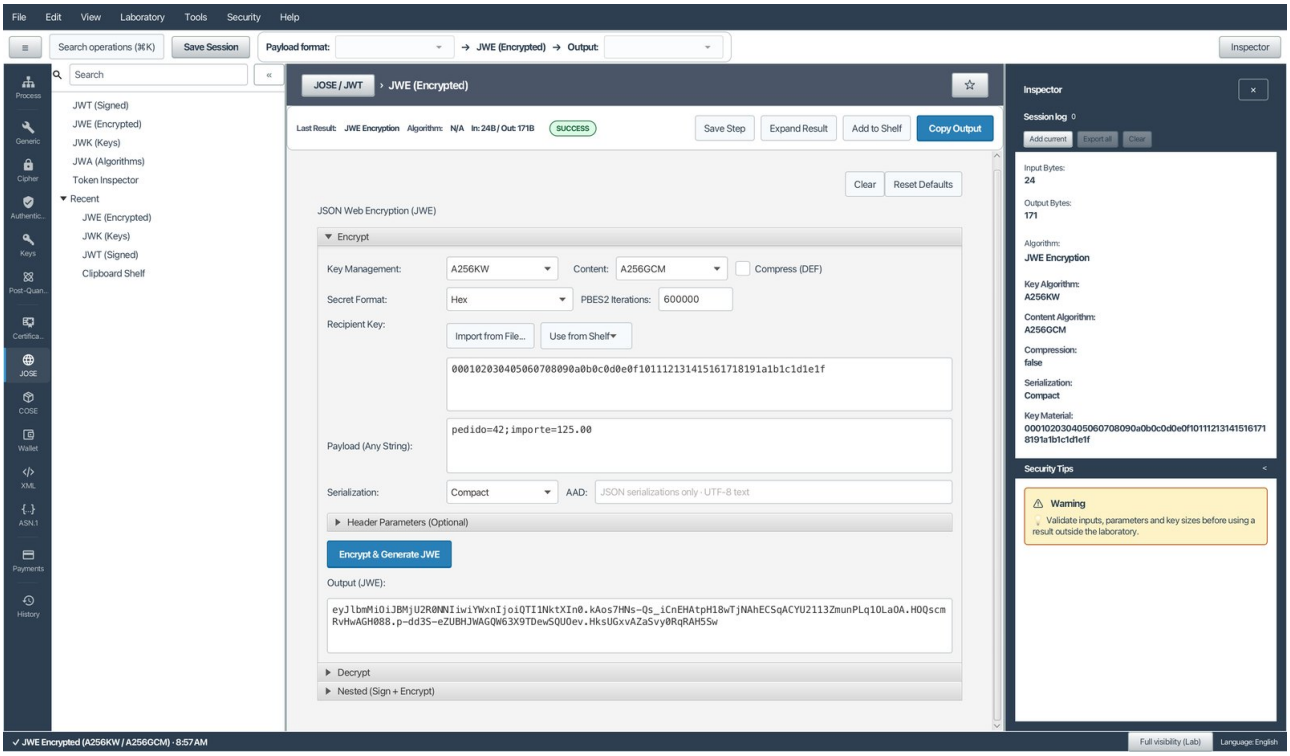


Decrypt: con la privada RSA se recuperan cabecera y payload

Repite con `Compress (DEF)`: la cabecera gana `"zip":"DEF"`. Con un payload tan corto el token crece (486 caracteres); la compresión compensa con textos largos y repetitivos.

Laboratorio B — AES Key Wrap con secreto en hexadecimal

1. `Key Management = A256KW`, `Content = A256GCM`, `Secret Format = Hex`.
2. **Recipient Key**: el secreto de 256 bits del material de laboratorio.
3. Cifra: 171 caracteres. El segundo segmento contiene la CEK envuelta (40 bytes).
4. **Decrypt** con el mismo secreto y `Secret Format = Hex`.

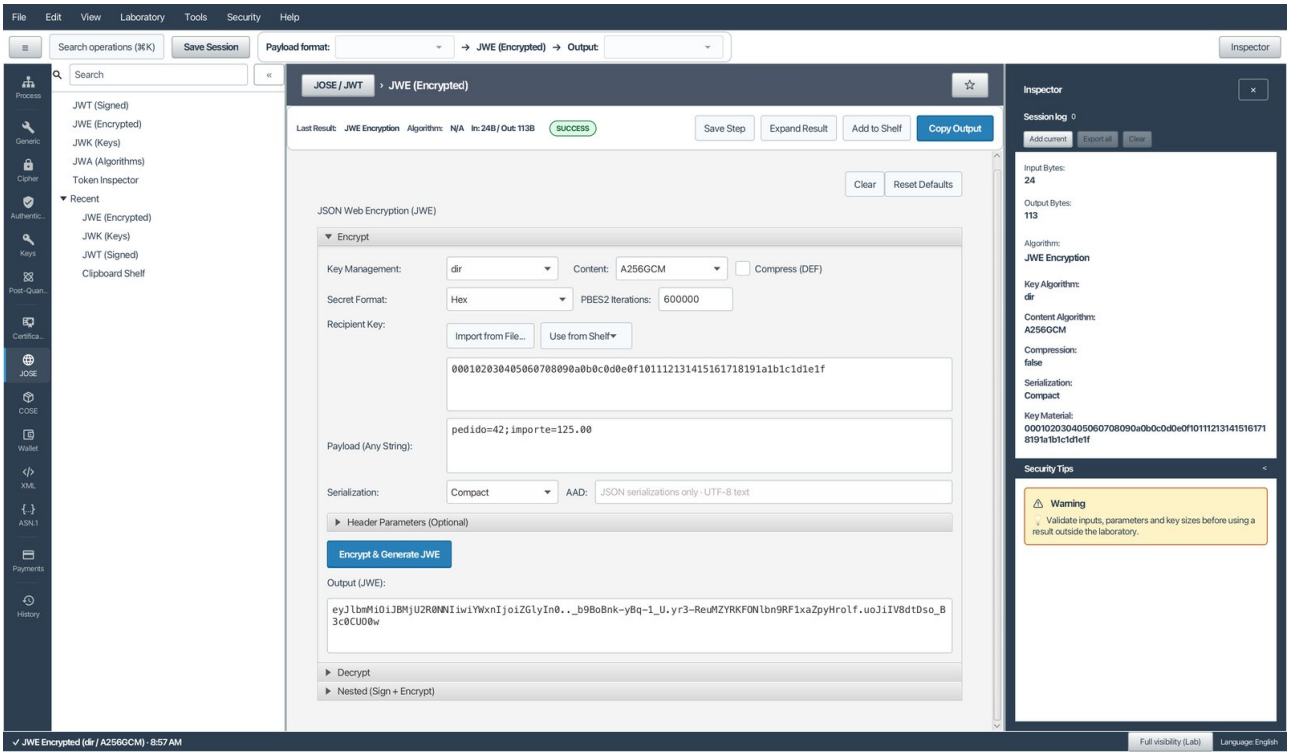


Laboratorio B: A256KW con un secreto de 256 bits leído como Hex

Prueba negativa: cambia `Secret Format` a `UTF-8` en Decrypt. El secreto pasa a medir 64 bytes y se rechaza por tamaño.

Laboratorio C — Cifrado directo (`dir`)

1. `Key Management = dir`, `Content = A256GCM`, `Secret Format = Hex`, secreto de 256 bits.
2. Cifra: 113 caracteres. El token contiene dos puntos seguidos (`..`): el segmento `encrypted_key` está vacío.
3. Cambia `Content` a `A128CBC-HS256` sin tocar la clave: sigue funcionando, porque ese método también pide 32 bytes.
4. Cambia a `A256CBC-HS512`: falla; necesita 64 bytes.



Laboratorio C: cifrado directo; el segmento encrypted_key queda vacío (..)

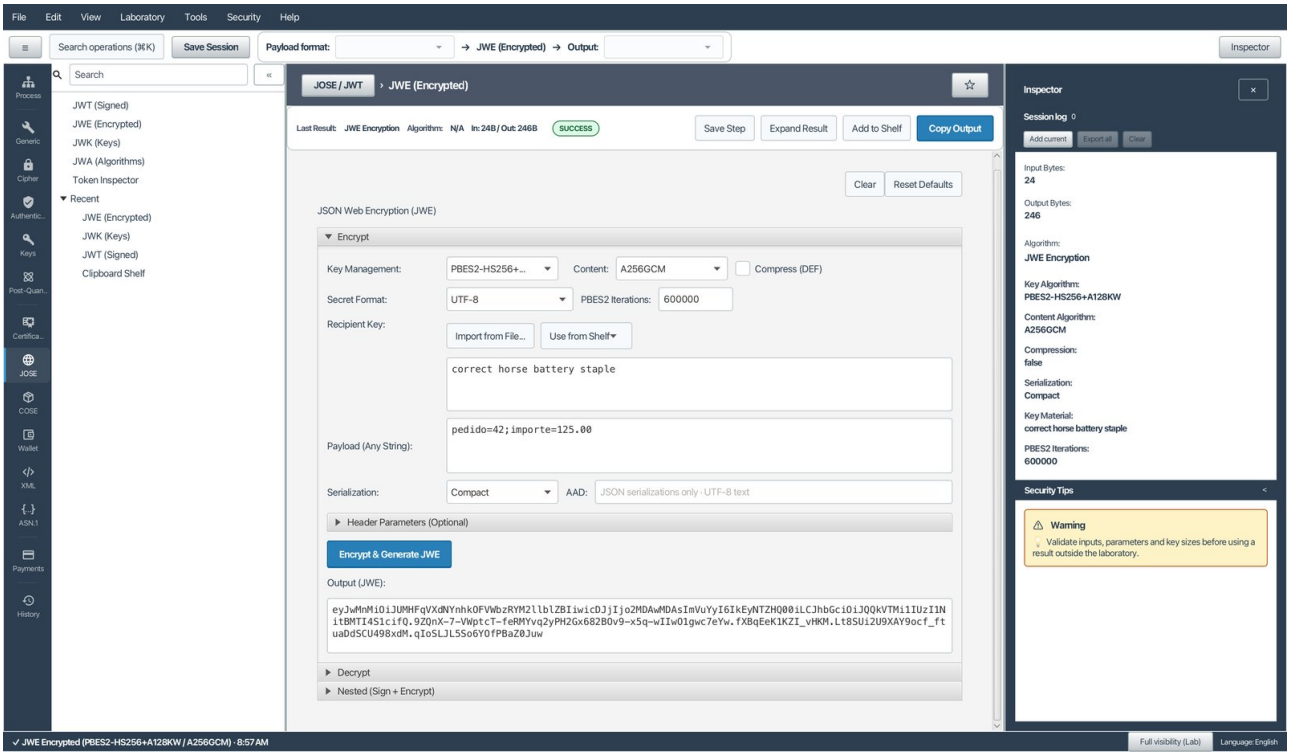
`dir` no tiene clave por mensaje: quien conoce el secreto descifra todos los tokens. Resérvalo para canales entre dos partes con rotación controlada.

Laboratorio D — Contraseña con PBES2

1. `Key Management = PBES2-HS256+A128KW`, `Secret Format = UTF-8`, `PBES2 Iterations = 600000`.
2. **Recipient Key**: `correct horse battery staple`.
3. Cifra y lleva el token a **Token Inspector**. La cabecera incluye:

```
{"alg": "PBES2-HS256+A128KW", "enc": "A256GCM", "p2c": 600000, "p2s": "5lVS46Fu74KIajQ6TkMjcw"}
```

4. Descifra con la misma contraseña. No hay que indicar las iteraciones: se leen de `p2c`.



Laboratorio D: PBES2 con contraseña y 600000 iteraciones PBKDF2

Las iteraciones encarecen cada intento de adivinar la contraseña. Menos de 1000 se rechaza en el formulario; por encima de 1 000 000 el descifrador rechaza el token para no prestarse a ataques de consumo de CPU.

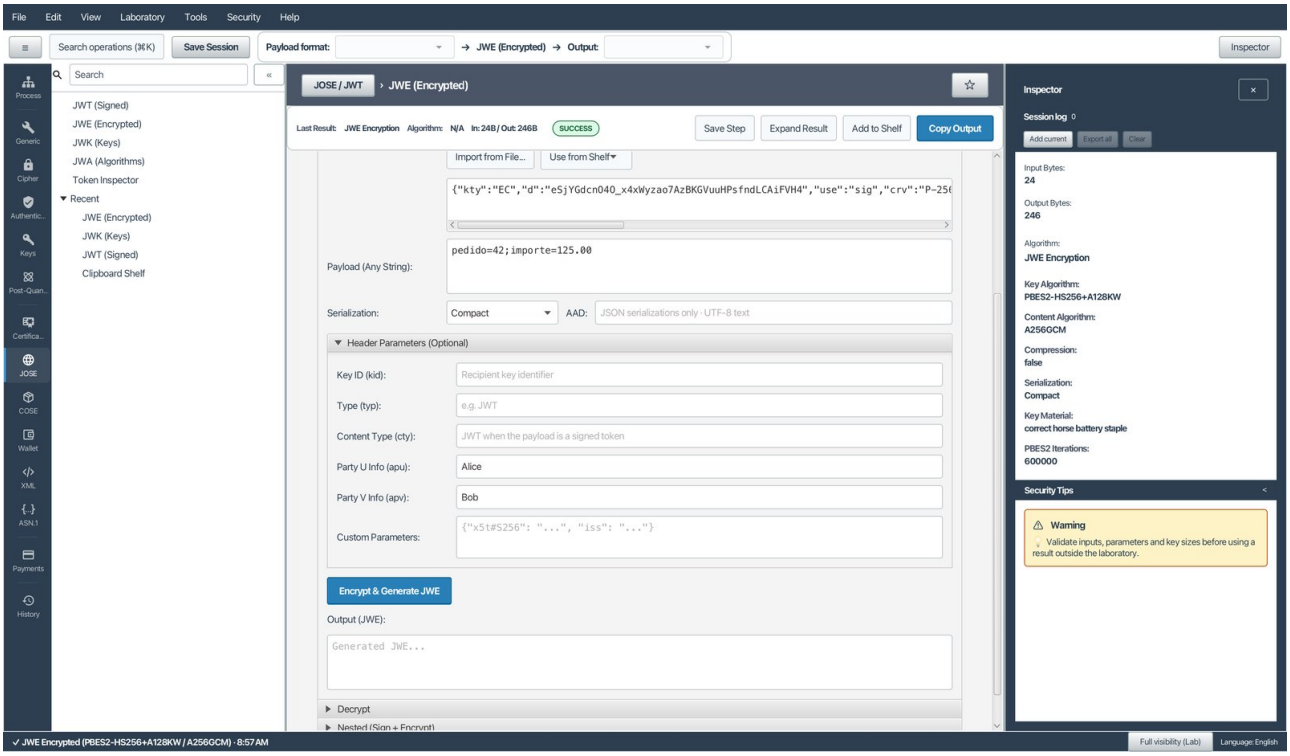
Laboratorio E — ECDH-ES con curva elíptica

1. En **JWK (Keys)** → **JWK Set (JWKS)** pulsa **New JWKS**, elige `ES256` y **Add New Key to Set**. Copia la JWK generada (necesitas el perfil de visibilidad de laboratorio para ver la privada).
2. En **Encrypt**: `Key Management = ECDH-ES+A256KW`, `Content = A256GCM`, pega la JWK en **Recipient Key**.
3. En **Header Parameters**: `apu = Alice`, `apv = Bob`.
4. Cifra e inspecciona. La cabecera lleva la clave efímera del emisor y las identidades en Base64URL:

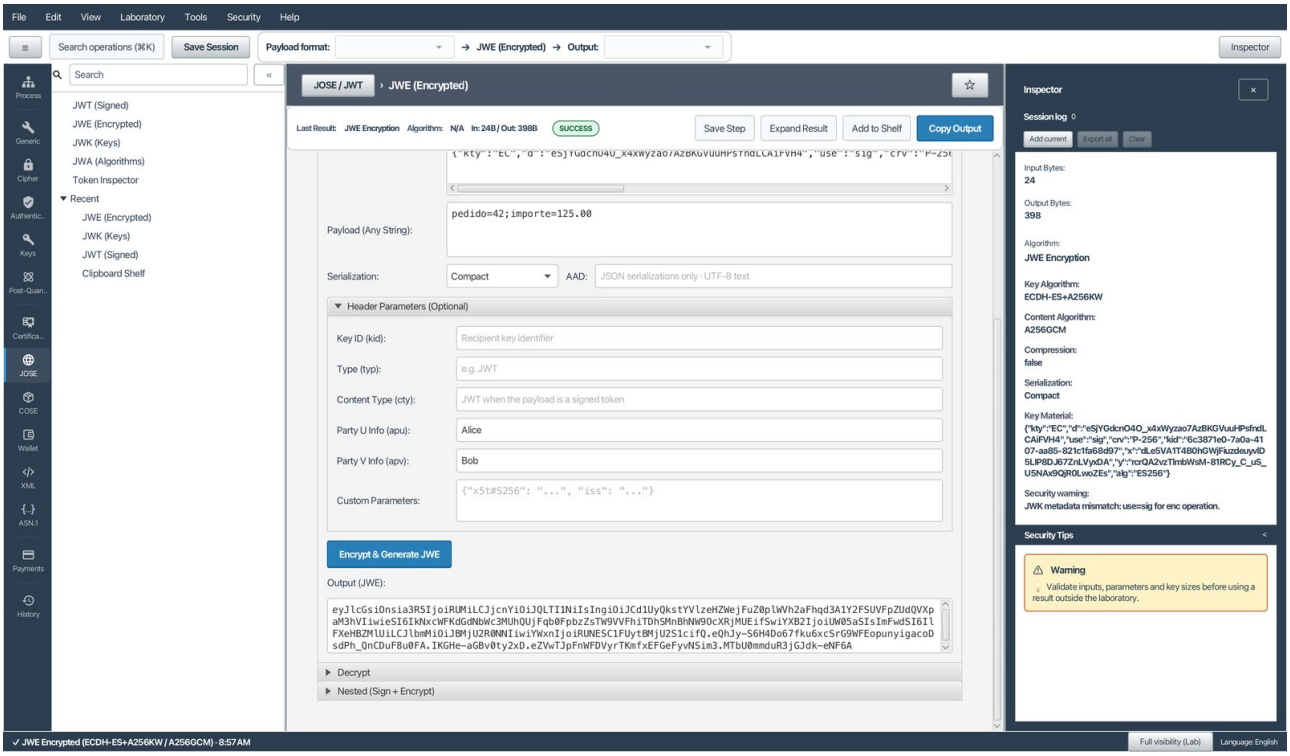
```
{"alg": "ECDH-ES+A256KW", "enc": "A256GCM", "apu": "QWxpY2U", "apv": "Qm9i",
 "epk": {"kty": "EC", "crv": "P-256", "x": "…", "y": "…"}}
```

5. Descifra con la misma JWK.

La clave generada lleva `use: sig`, así que el resultado anota el aviso *use=sig for enc operation* (sección 3.2): es informativo y no impide la operación.



Laboratorio E: ECDH-ES+A256KW con la clave EC generada; apu y apv en Header Parameters



El token lleva la clave efímera del emisor (epk) en la cabecera protegida

Repite con `ECDH-ES` a secas: el segundo segmento queda vacío, porque la CEK se deriva directamente del acuerdo. Para X25519, genera la clave con `ECDH-ES` en la rotación de JWKS (crea una OKP X25519) y úsala igual; `epk` pasa a ser `{"kty":"OKP","crv":"X25519","x":"…"}`.

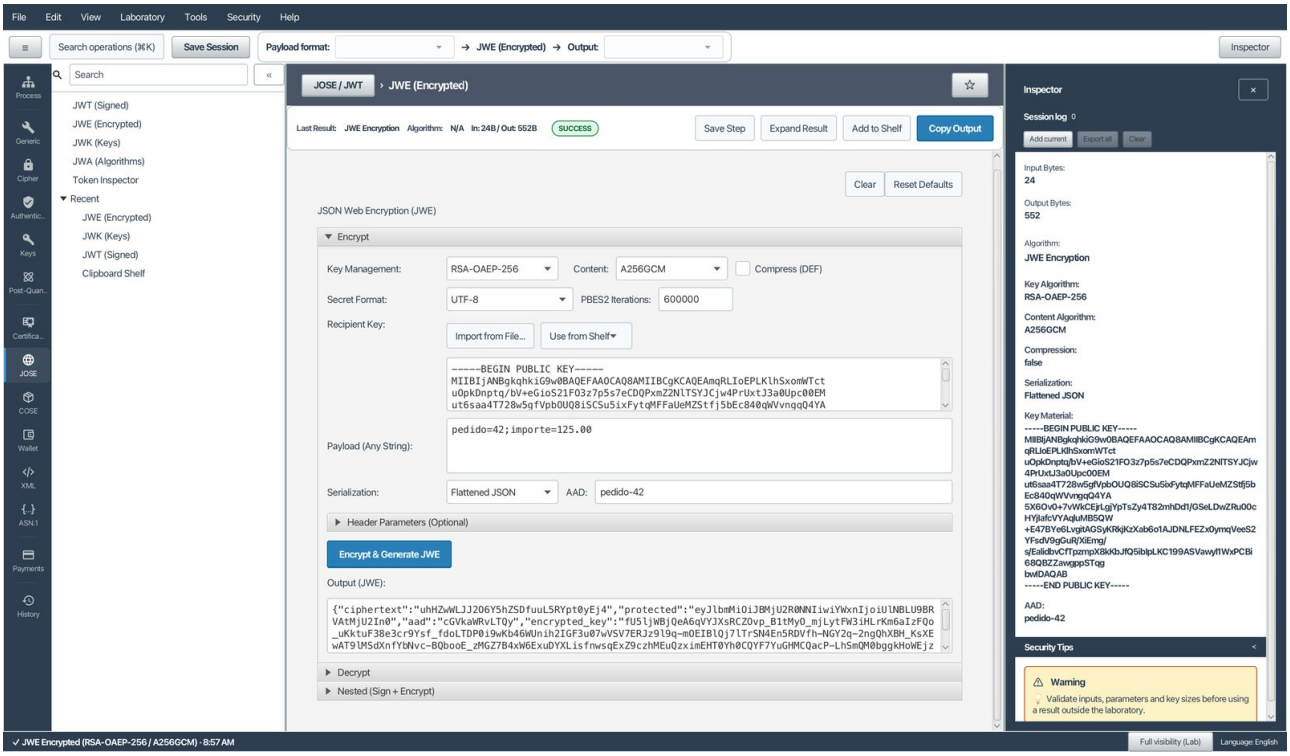
Laboratorio F — AAD en Flattened JSON

1 Parte del laboratorio A y cambia `Serialization = Flattened JSON` y `AAD = pedido-42`.

2 Cifra. El resultado es un objeto JSON con el miembro `"aad":"cGVkaWRvLTQy"`.

3 Descifra: correcto.

4 Edita en el token un carácter del valor `aad` y descifra de nuevo: falla, aunque el ciphertext no ha cambiado.



Laboratorio F: Flattened JSON con AAD; el miembro aad va autenticado

Laboratorio G — Tres destinatarios

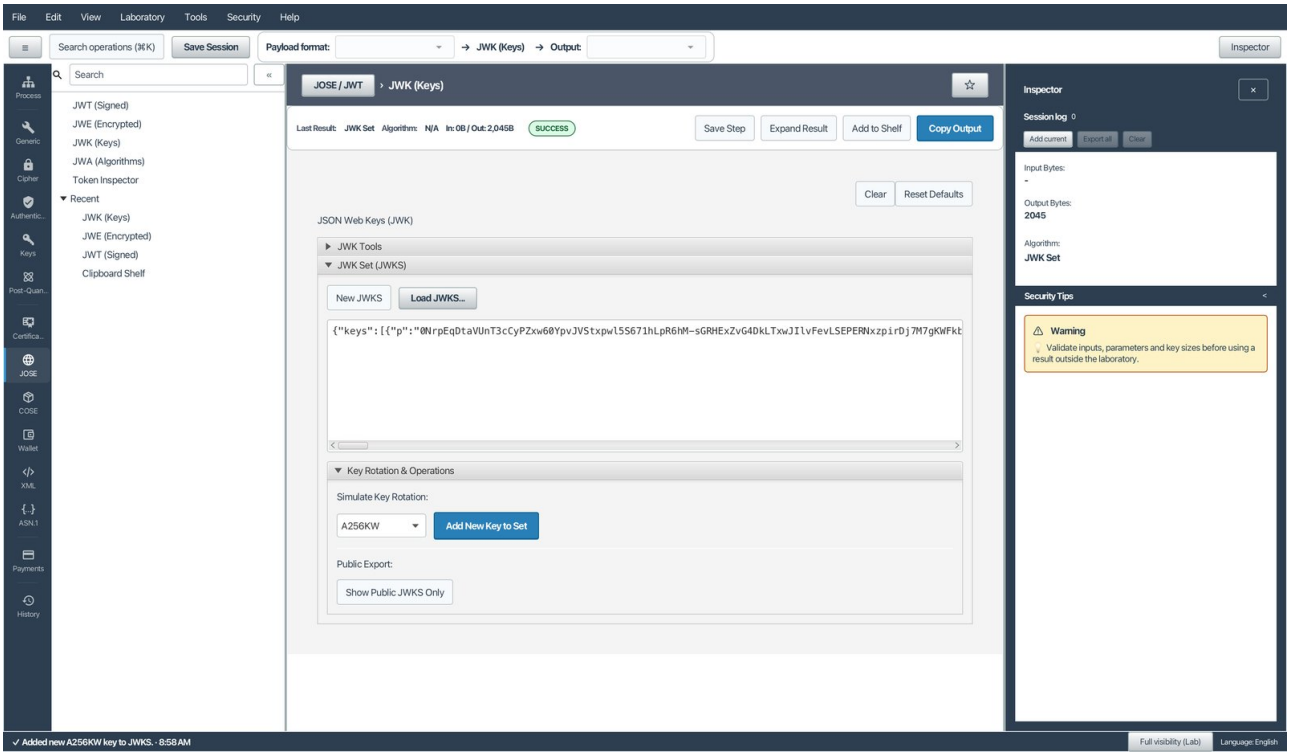
1 En **JWK Set (JWKS)**: **New JWKS** y añade tres claves: `RSA-OAEP`, `ECDH-ES+A256KW` (genera una X25519) y `A256KW` (acepta el aviso de clave simétrica).

2 Copia el JWKS completo a **Recipient Key**, con `Serialization = General JSON`. Las claves generadas ya declaran su `alg`.

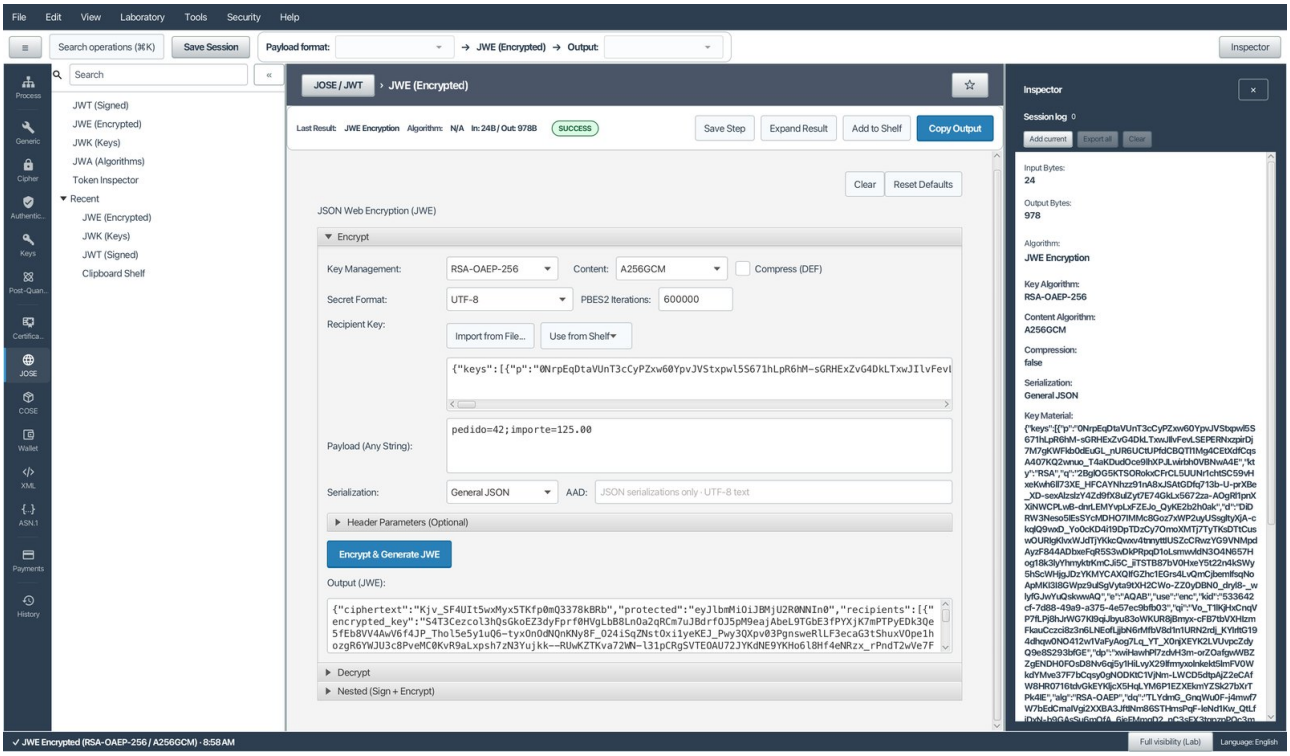
3 Cifra. El resultado tiene tres entradas en `recipients`.

4 Descifra con cada clave por separado: la JWK RSA privada, la JWK X25519 privada y el secreto `oct` (pásalo antes por **JWK** → **Secret** y usa `Secret Format = Hex`). El estado indica *recipient 1/3*, *2/3* y *3/3*.

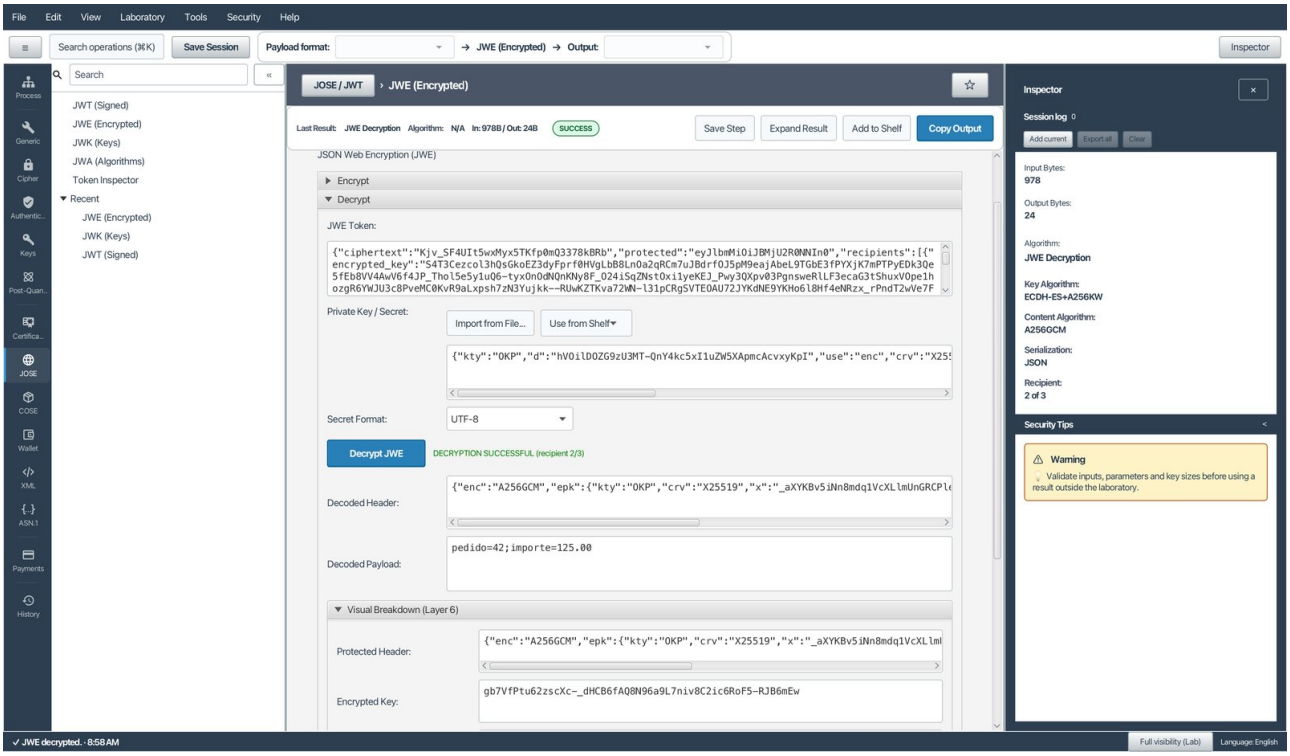
En la rotación elige `ECDH-ES+A256KW` y no `ECDH-ES` a secas: el acuerdo directo no puede compartir la CEK con otros destinatarios.



Laboratorio G: un JWKS con tres claves (RSA, X25519 y oct); cada una declara su alg



General JSON: una entrada por destinatario en recipients y un solo ciphertext



Cada destinatario descifra con su clave: el estado indica cuál encajó (recipient 2/3)

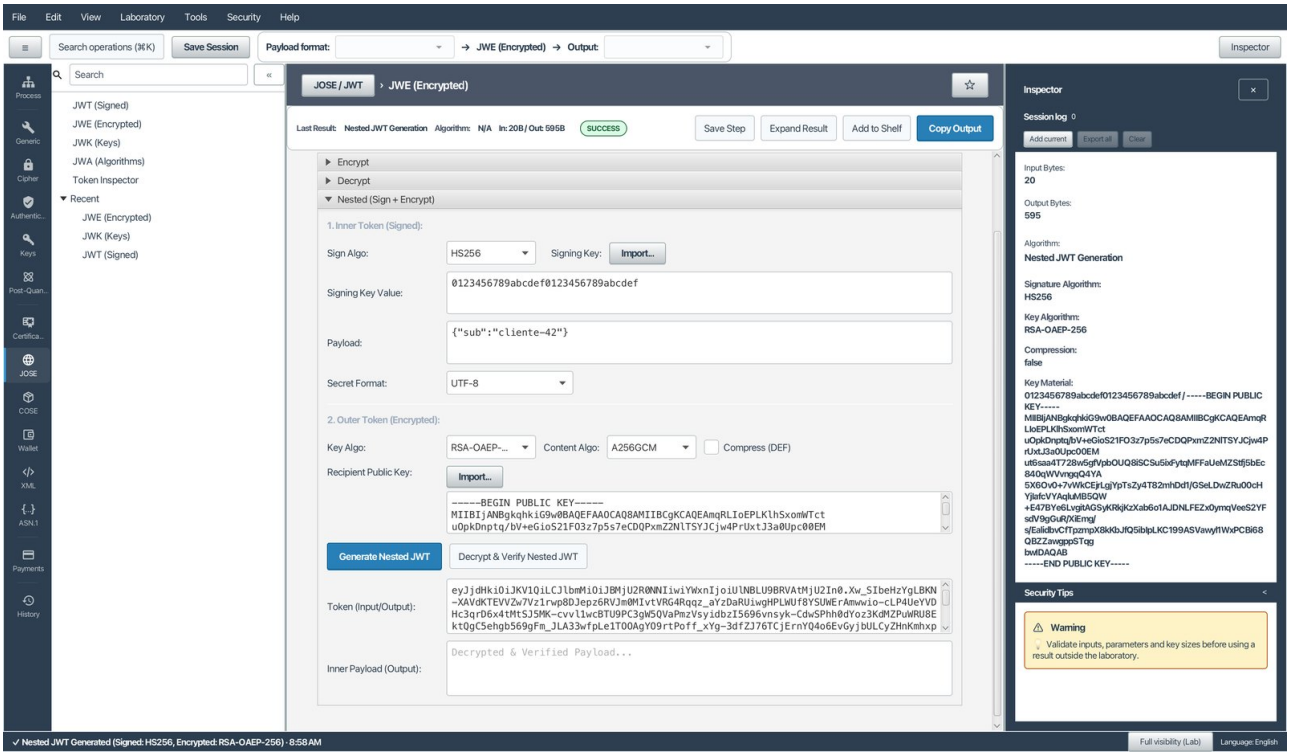
2.9 Nested (Sign + Encrypt)

Firma primero y cifra después. El resultado es un JWE compacto con `cty: JWT` cuyo contenido es un JWS.

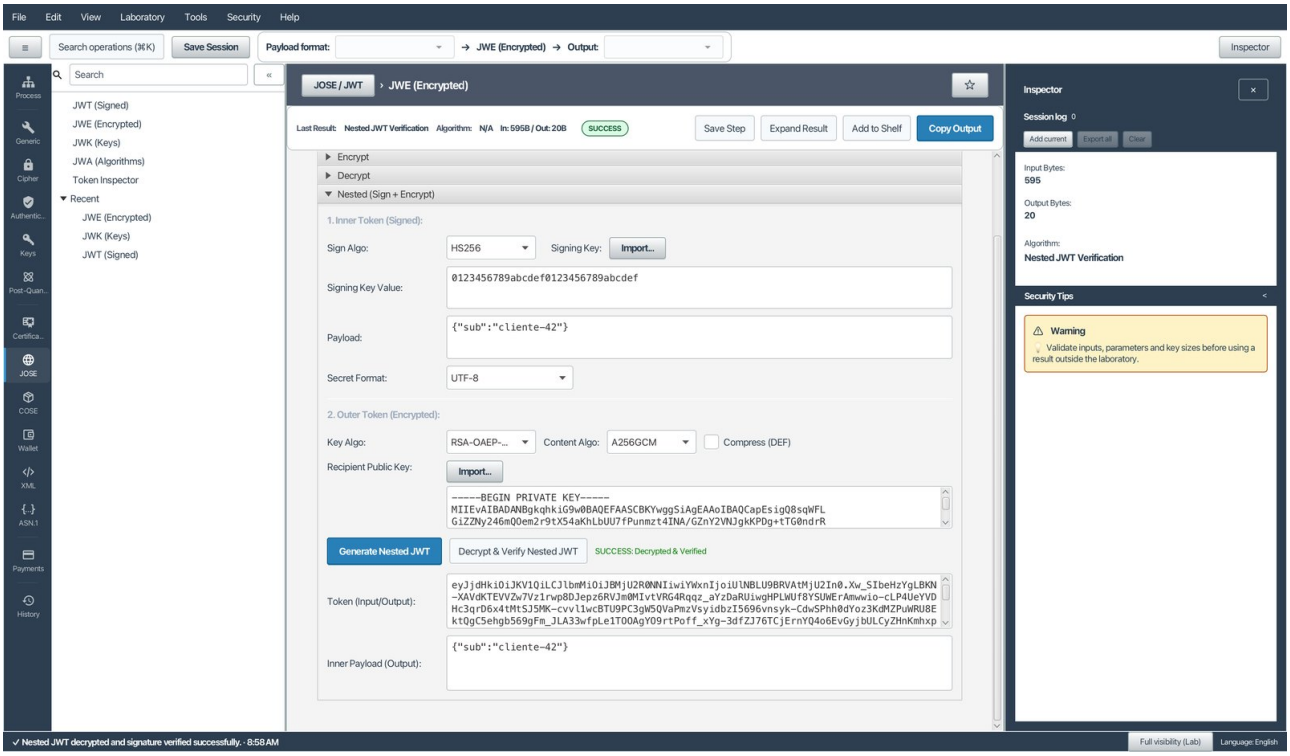
Bloque	Controles
1. Inner Token (Signed)	**Sign Algo** (los mismos quince que JWT), **Signing Key Value**, **Payload** (JSON de claims), **Secret Format**
2. Outer Token (Encrypted)	**Key Algo**, **Content Algo**, **Compress (DEF)**, **Recipient Public Key**
Acciones	**Generate Nested JWT** · **Decrypt & Verify Nested JWT**
Salidas	**Token (Input/Output)** · **Inner Payload (Output)**

- **Key Algo** lista los diecinueve algoritmos, pero aquí solo funcionan `RSA*`, `ECDH-ES*` y `dir`. El resto devuelve *Unsupported JWE algorithm for Nested JWT*.
- **Secret Format** se aplica a la vez al secreto HMAC de firma y a la clave `dir`.
- Para **verificar**, el mismo formulario cambia de papel: el campo de clave de cifrado debe contener la **privada** del destinatario (o el secreto `dir`), y el de firma, la clave de verificación. Sirve la privada del firmante: se deriva la pública.
- El orden de comprobación es fijo: primero descifrar, después verificar la firma interior. Si la firma falla: *Nested JWT decrypted, but inner signature verification failed.*

Perfil reproducible: `HS256` con el secreto `0123456789abcdef0123456789abcdef` (UTF-8), `RSA-OAEP-256` + `A256GCM`, payload `{"sub":"cliente-42"}`.



Nested: se firma con HS256 y el JWS resultante se cifra con RSA-OAEP-256



Decrypt & Verify: primero se descifra y después se verifica la firma interior

Parte 3 — JWK (Keys)

3.1 Anatomía de una JWK

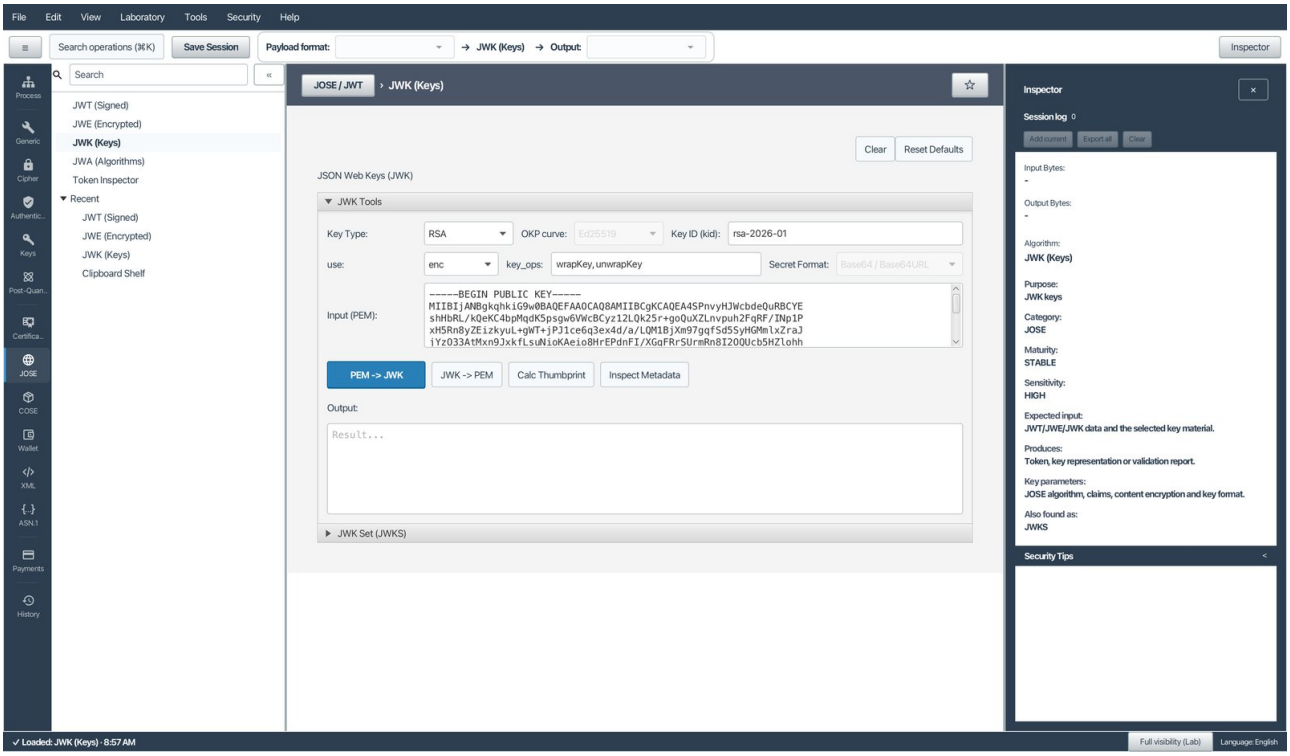
kty	Parámetros públicos	Parámetros privados	Algoritmos típicos
RSA	n, e	d, p, q, dp, dq, qi	RS/PS, RSA-OAEP
EC	crv, x, y	d	ES256/384/512, ES256K, ECDH-ES
OKP	crv, x	d	EdDSA (Ed25519/Ed448), ECDH-ES (X25519/X448)
oct	—	k	HS*, A*KW, A*GCMKW, dir

Metadatos comunes: `kid` (identificador), `use` (`sig` o `enc`), `key_ops` (operaciones permitidas) y `alg` (algoritmo previsto).

3.2 Panel JWK Tools

Control	Qué hace
Key Type	`RSA`, `EC`, `OKP` u `OCT`. Debe coincidir con la clave que pegas
OKP curve	`Ed25519`, `Ed448`, `X25519`, `X448`. Solo activo con `OKP`
Key ID (kid)	Identificador. Vacío = se usa el thumbprint
use	`sig` o `enc`
key_ops	Lista separada por comas
Secret Format	`UTF-8`, `Hex` o `Base64 / Base64URL`. Solo activo con `OCT`; por defecto Base64
Input	PEM, JWK o, con `OCT`, el secreto
PEM → JWK	Convierte a JWK
JWK → PEM	Convierte a PEM
Calc Thumbprint	Thumbprint SHA-256 según RFC 7638
Inspect Metadata	Resume `kty`, `kid`, `use`, `key_ops`, `alg`

Al elegir `OCT`, las etiquetas cambian: **Input** pasa a pedir un secreto y los botones se llaman **Secret → JWK** y **JWK → Secret**.



JWK Tools con una clave pública PEM y sus metadatos: kid, use y key_ops

PEM → JWK

Acepta todos los formatos asimétricos de la Parte 1. El resultado depende de lo que pegues:

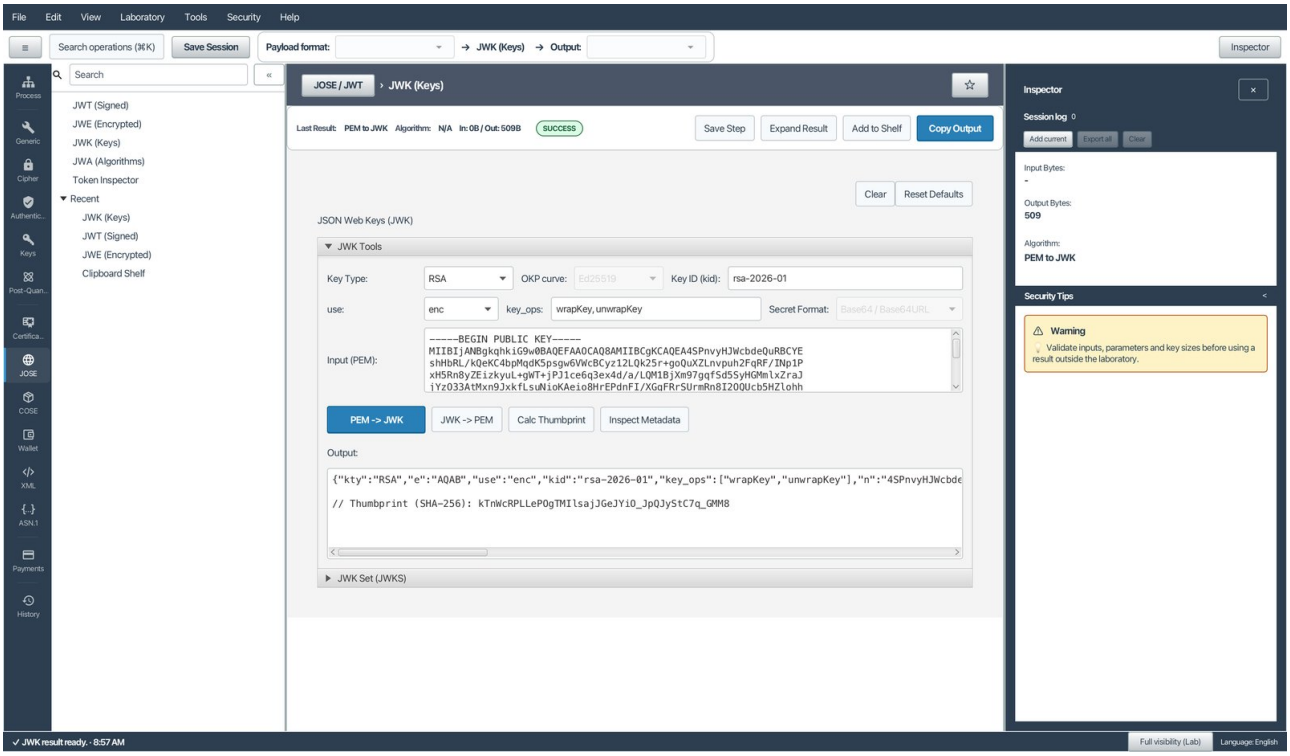
- Pública o certificado → JWK pública.
- Privada → JWK **privada**, con `d` y, en RSA, los factores CRT.

La salida añade siempre una línea de comentario con el thumbprint. Con `kid = rsa-2026-01`, `use = enc` y `key_ops = wrapKey, unwrapKey`:

```
{"kty":"RSA","e":"AQAB","use":"enc","kid":"rsa-2026-01","key_ops":["wrapKey","unwrapKey"],"n":"…
"}

// Thumbprint (SHA-256): kTnWcRPLLePOgTMIlsajJGeJYiO_JpQJyStC7q_GMM8
```

Ese valor es el de clave-publica-laboratorio.pem; al copiar la JWK a otro sitio no incluyas la línea de comentario.



PEM → JWK: la JWK pública con sus metadatos y el thumbprint SHA-256

Si **Key Type** no coincide con la clave real, o `use` y `key_ops` se contradicen, la respuesta es deliberadamente genérica: *Material de clave no válido o no compatible.* Revisa esos tres selectores.

`use` y `key_ops`

`use` siempre se escribe, porque el selector tiene valor (por defecto `sig`). `key_ops` solo si lo rellenas. Valores admitidos:

```
sign, verify, encrypt, decrypt, wrapKey, unwrapKey, deriveKey, deriveBits
```

Ambos deben ser coherentes (RFC 7517 §4.3):

`use`	`key_ops` **compatibles**
`sig`	`sign`, `verify`
`enc`	`encrypt`, `decrypt`, `wrapKey`, `unwrapKey`, `deriveKey`, `deriveBits`

Para una clave de destinatario JWE elige `use = enc`. Para ECDH-ES lo natural es `key_ops = deriveKey, deriveBits`; para RSA-OAEP, `wrapKey, unwrapKey`.

Estos metadatos son **informativos**: CryptoCarver no bloquea una operación porque contradiga `use` o `key_ops`, pero lo anota en el resultado como *Security warning*. Si cifras con una JWK marcada `use: sig` y `key_ops: ["verify"]` verás:

```
Metadatos JWK incompatibles: use=sig for enc operation; key_ops does not include encrypt.
```

Un verificador o un HSM en producción sí puede rechazarla.

Curvas OKP

El selector de curva no transforma la clave que pegas —la curva se detecta sola—; determina qué se genera en la rotación de JWKS, y mantiene coherentes los demás selectores:

Curva	`use` que fija	Algoritmo de rotación que fija
`Ed25519`, `Ed448`	`sig`	`EdDSA`
`X25519`	`enc`	`ECDH-ES`
`X448`	`enc`	`ECDH-ES-X448`

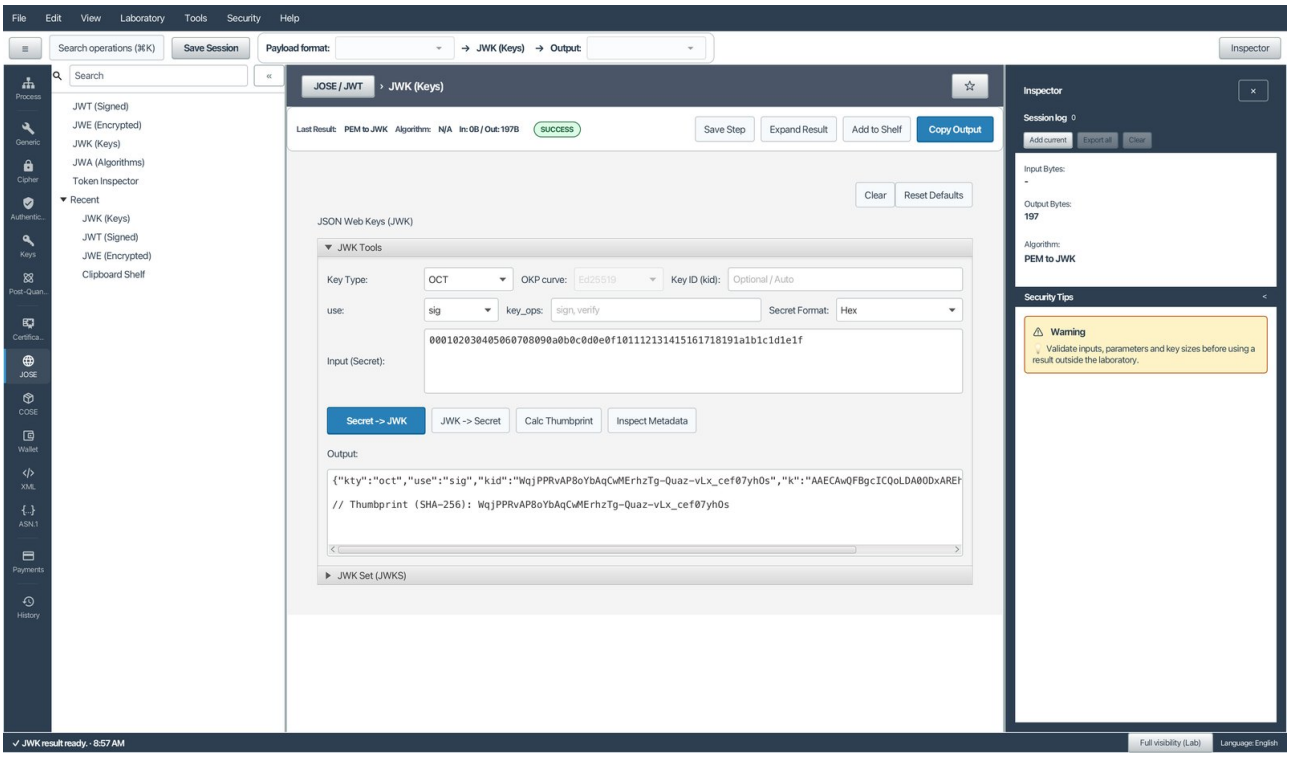
Recuerda la separación de funciones: las curvas Ed firman y las X acuerdan claves. Una clave Ed25519 no sirve como destinatario de `ECDH-ES`.

Secret → JWK (tipo OCT)

Elige **Secret Format**, pega el secreto y pulsa **Secret → JWK**. Con el secreto de 256 bits del laboratorio —en `Hex` o en `Base64 / Base64URL`, cada uno con su formato seleccionado—, `use = sig` y `kid` vacío:

```
{"kty":"oct","use":"sig","kid":"WqjPPRvAP8oYbAqCwMErhzTg-Quaz-vLx_cef07yhOs","k":"AAECAwQFBgcICQ
oLDA0ODxAREhMUFRYXGBkaGxwdHh8"}
```

El thumbprint `WqjPPRvAP8oYbAqCwMErhzTg-Quaz-vLx_cef07yhOs` es determinista: debe salirte exactamente ese.



Secret → JWK: clave simétrica oct; el thumbprint es determinista

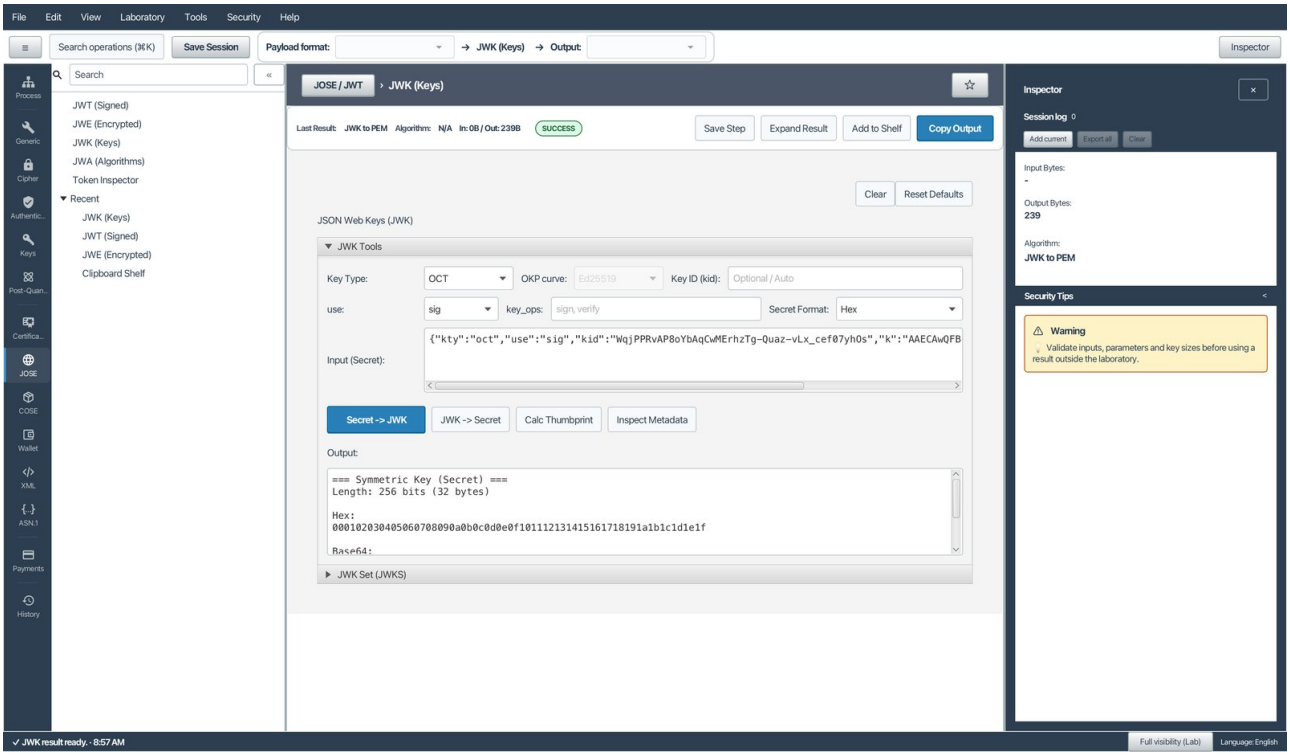
El formato nunca se adivina: se aplica el seleccionado. Es la misma regla que en JWE, con una diferencia: aquí el valor por defecto es `Base64 / Base64URL`, la forma en que una JWK guarda su `k`. Prueba negativa: pega el valor hexadecimal dejando Base64 seleccionado. Se acepta sin error —los dígitos hexadecimales también son caracteres Base64 válidos— pero el thumbprint ya no coincide, porque es otra clave. Con `Hex` y un texto que no sea hexadecimal, la conversión se rechaza.

Toda JWK `oct` es material secreto: se trata como privada a efectos de visibilidad.

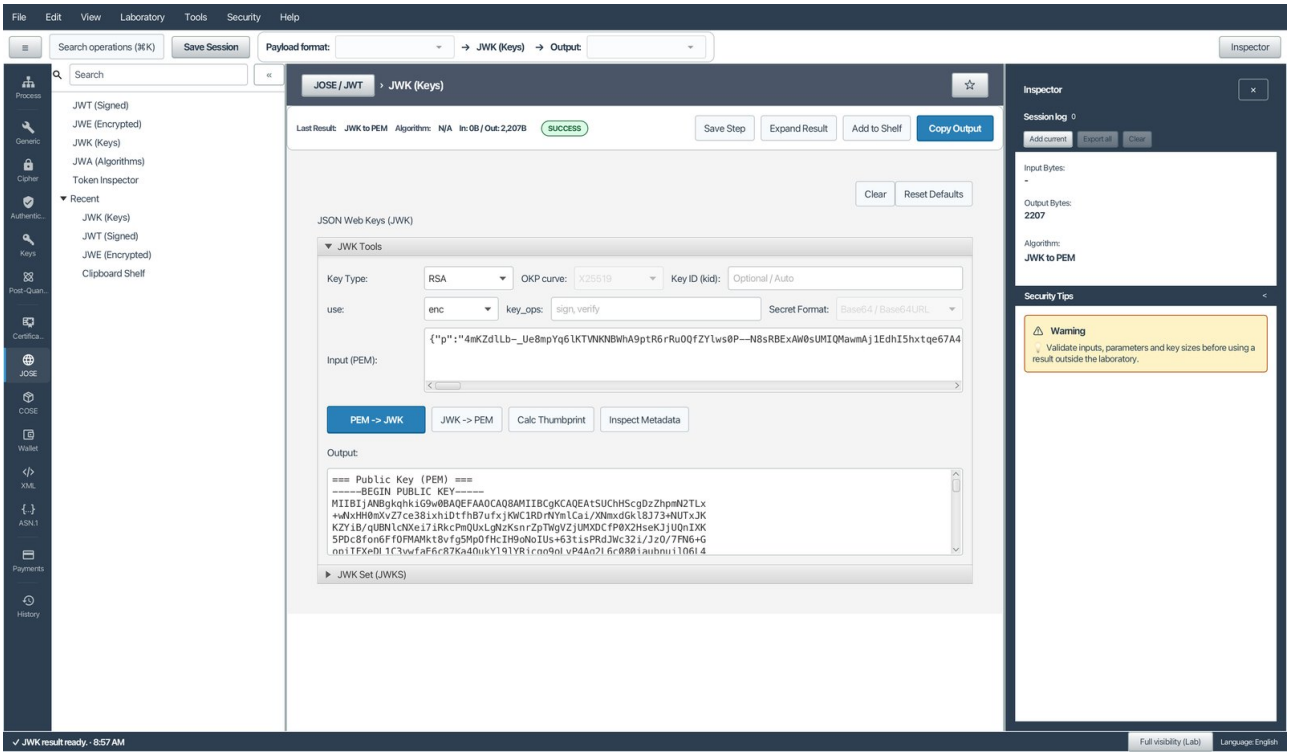
JWK → PEM y JWK → Secret

Entrada	Salida
JWK RSA / EC / OKP pública	`PUBLIC KEY` (SubjectPublicKeyInfo)
JWK RSA / EC / OKP privada	Además, `PRIVATE KEY` (PKCS#8)
JWK `oct`	Longitud en bits y bytes, y el secreto en Hex, Base64 y Base64URL

Incluye `secp256k1` (ES256K) y las cuatro curvas OKP. La salida `oct` en Hex es la forma cómoda de llevar el secreto a un campo con `Secret Format = Hex`.



JWK → Secret: el mismo secreto en Hex, Base64 y Base64URL

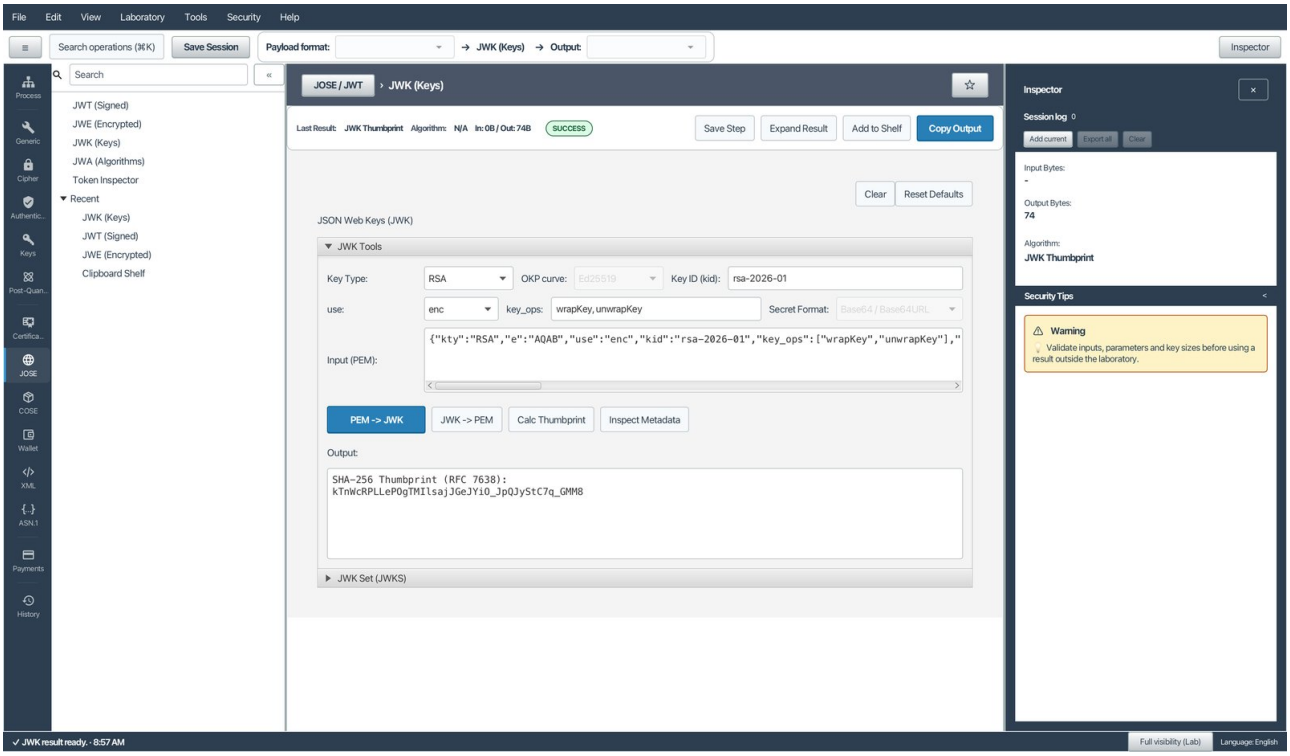


JWK → PEM: el par RSA generado, exportado como PUBLIC KEY y PRIVATE KEY

Calc Thumbprint

Acepta una JWK o un PEM. El thumbprint RFC 7638 se calcula solo sobre los parámetros públicos obligatorios, en orden canónico, de modo que:

- no depende de `kid`, `use`, `key_ops` ni `alg`;
- es el mismo para la pública y para su privada;
- coincide con el valor `jkt` que usa DPoP en el claim `cnf`.

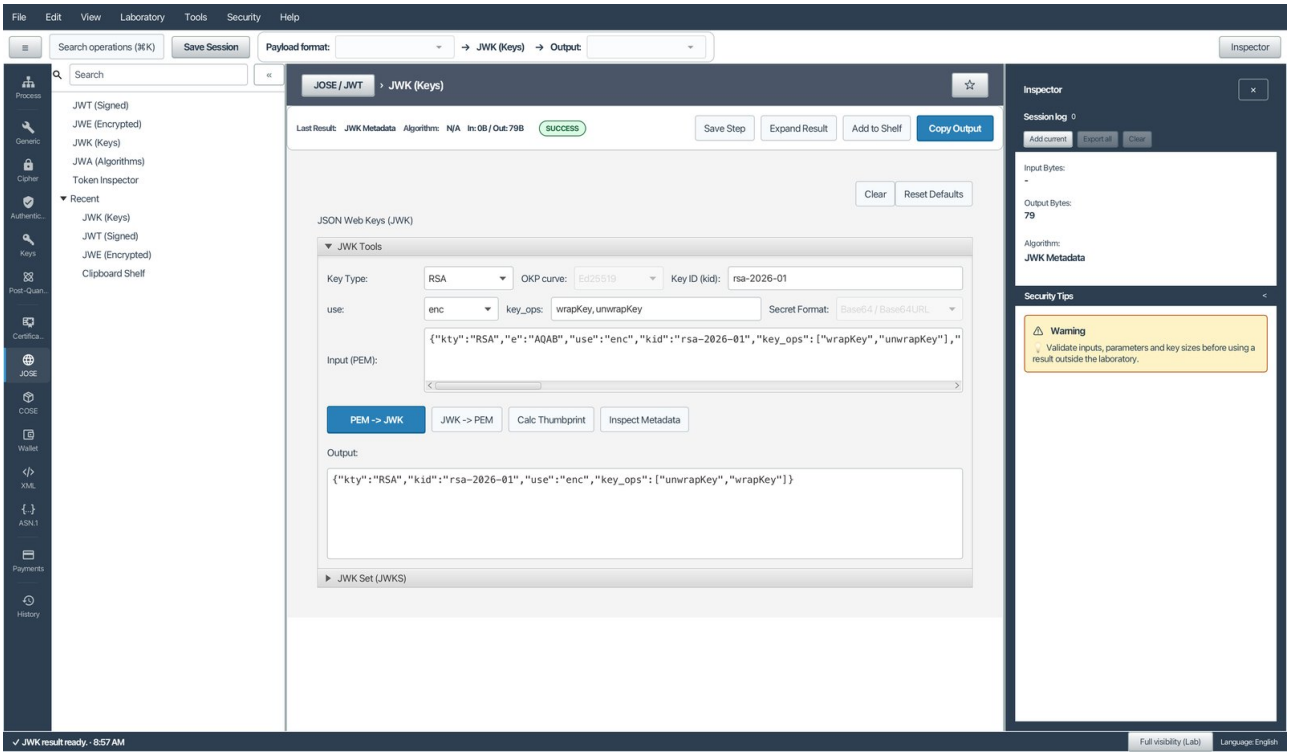


Calc Thumbprint: huella RFC 7638, independiente de kid, use y key_ops

Inspect Metadata

Solo con JWK en JSON. Devuelve un resumen sin material de clave, útil para revisar una clave antes de usarla:

```
{"kty":"RSA","kid":"rsa-2026-01","use":"enc","key_ops":["unwrapKey","wrapKey"],"alg":"RSA-OAEP-2
56"}
```



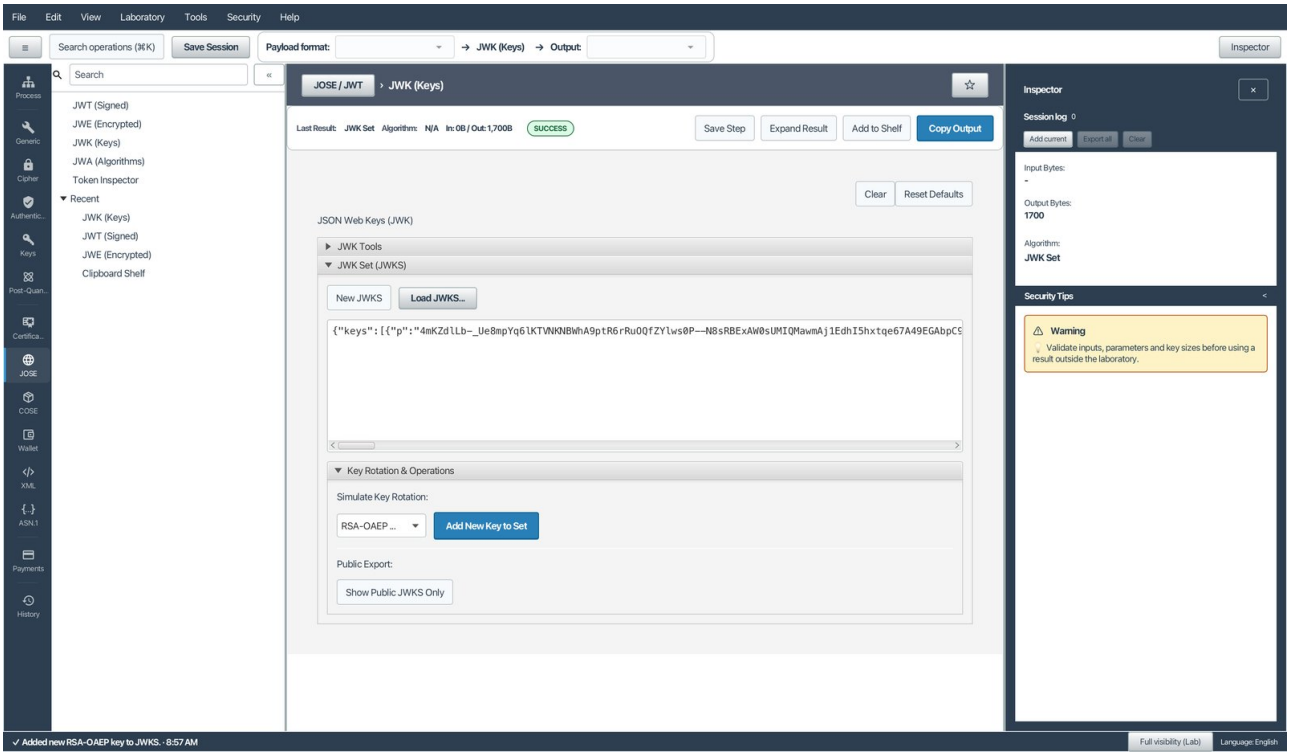
Inspect Metadata: resumen de kty, kid, use y key_ops sin material de clave

3.3 Panel JWK Set (JWKS)

Control	Qué hace
New JWKS	Empieza con `{"keys": []}`
Load JWKS…	Carga un archivo `.json`
Área de texto	El conjunto, editable a mano
Simulate Key Rotation	Algoritmo de la clave a generar
Add New Key to Set	Genera una clave y la añade
Show Public JWKS Only	Abre un diálogo solo con las partes públicas

Qué genera cada algoritmo

Cada clave nueva recibe un `kid` UUID aleatorio, su `alg` y un `use` deducido del algoritmo. Si el campo **key_ops** de *JWK Tools* está relleno, también se aplica.



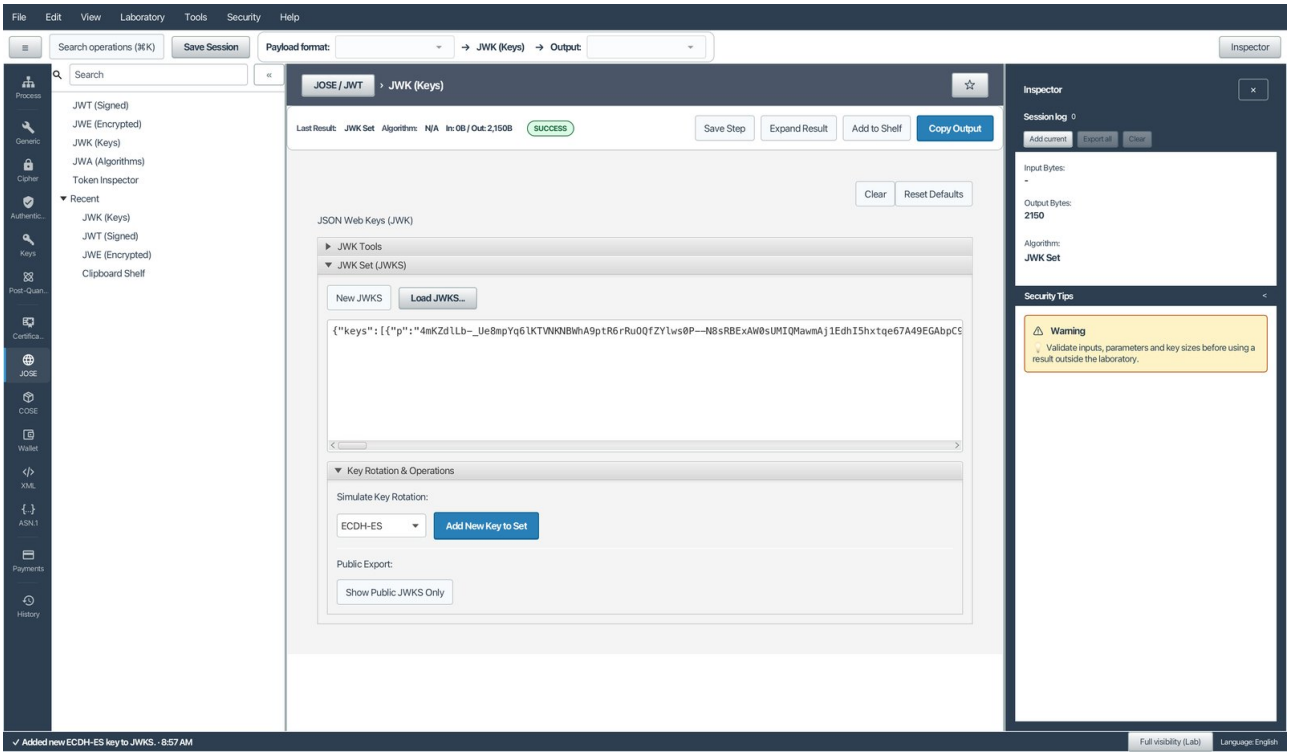
Add New Key to Set con RSA-OAEP: genera un par RSA-2048 con kid aleatorio y use=enc

Algoritmo elegido	Clave generada	use
RS256/384/512, PS256/384/512	RSA 2048	sig
ES256 / ES384 / ES512	EC P-256 / P-384 / P-521	sig
ES256K	EC secp256k1	sig
EdDSA	OKP Ed25519, o Ed448 si esa es la curva seleccionada	sig
RSA1_5, RSA-OAEP	RSA 2048	enc
ECDH-ES, ECDH-ES+A128KW/A192KW/A256KW	OKP X25519, o X448 si esa es la curva seleccionada	enc
ECDH-ES-X448	OKP X448, con `alg: ECDH-ES`	enc
HS256 / HS384 / HS512	`oct` de 256 / 384 / 512 bits	sig

Algoritmo elegido	Clave generada	use
A128KW / A256KW	oct de 128 / 256 bits	enc
A128GCM / A256GCM	oct de 128 / 256 bits	enc
dir	oct de 256 bits	enc

Notas:

- Las claves de ECDH-ES son **OKP**, no EC. Para un destinatario ECDH sobre P-256, genera una ES256 y úsala en JWE: la curva es la misma. Si quieres publicarla con el propósito correcto, vuelve a pasarla por **PEM** → **JWK** con use = enc.
- No hay entradas RSA-OAEP-256/384/512. Una clave RSA-OAEP sirve igual para ellas como destinatario único; en multi-destinatario se usará el alg que declare la JWK, así que edítalo a mano si quieres SHA-2.



Con ECDH-ES se genera un par OKP X25519; la curva y use=enc se ajustan solos

Claves simétricas en un JWKS

Al añadir HS*, A* o dir aparece una advertencia: un JWKS publicado en .well-known/jwks.json con una clave oct entrega el secreto a cualquiera. Acepta solo si el conjunto es privado.

Exportación pública

Show Public JWKS Only elimina d, los factores RSA y las claves oct enteras. Un conjunto con una RSA y una oct se reduce a una sola entrada. Ese diálogo es lo que se publica; el área de texto conserva las privadas para que puedas seguir firmando y descifrando.

Visibilidad de material privado

Las salidas que contienen privadas respetan el perfil de visibilidad de secretos:

Perfil	Lo que muestra
Laboratorio completo	El material íntegro
Enmascarado	`***MASKED***`
Redactado	Un aviso de que está oculto

Fuera del perfil de laboratorio, copiar, cortar y el menú contextual quedan bloqueados en esas áreas. El JWKS real se conserva en memoria aunque no se muestre: **Add New Key to Set** y **Show Public JWKS Only** siguen funcionando.

3.4 Laboratorio de rotación completo

Objetivo: rotar la clave de cifrado de un servicio sin interrumpir a los emisores.

1. **New JWKS**. Añade una clave `RSA-OAEP`: será la clave vigente. Anota su `kid`.
2. **Show Public JWKS Only** y copia el resultado: es lo que verían los emisores.
3. En **JWE → Encrypt**, cifra el payload con esa JWK pública, `RSA-OAEP-256`, y pon su `kid` en **Key ID (kid)**.
4. Vuelve al JWKS y añade una segunda `RSA-OAEP`: la clave nueva. Ahora hay dos.
5. Cifra un segundo token con la clave nueva y su `kid`.
6. En **Token Inspector**, lee el `kid` de cada token: te dice qué privada necesita.
7. Descifra cada uno copiando al campo de clave **solo la JWK privada correspondiente**. Si pegas el JWKS entero: *The JWKS holds 2 keys; paste the single JWK to use.*
8. Cuando ya no circulen tokens con la clave antigua, bórrala del área de texto.

Durante la ventana de coexistencia el receptor conserva ambas privadas y elige por `kid`. El `kid` es un selector, no una prueba de confianza: nunca aceptes un JWKS, `jku` o `x5u` que venga indicado por el propio token.

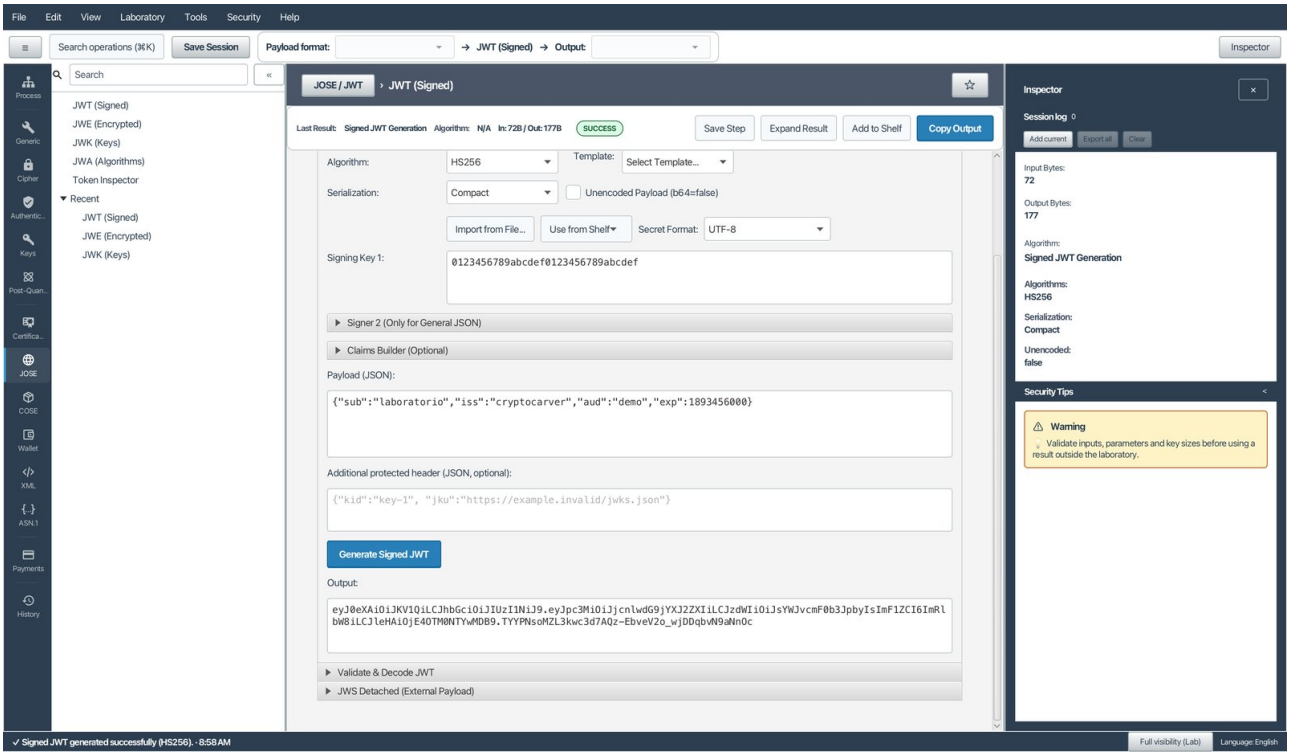
Parte 4 — Referencia de JWS y JWT

4.1 Generate JWT

Control	Opciones
Algorithm	`HS256/384/512`, `RS256/384/512`, `ES256`, `ES256K`, `ES384`, `ES512`, `PS256/384/512`, `EdDSA`, `none`
Template	*OAuth2 Access Token (JWT)*, *OIDC ID Token*, *DPoP Proof*, *Custom (Empty)*. Rellena el payload con `iat`/`exp` actuales
Serialization	`Compact`, `Flattened JSON`, `General JSON`
Unencoded Payload (b64=false)	RFC 7797: firma el payload sin Base64URL y añade `b64:false` y `crit:["b64"]`. El resultado sale con payload separado
Signing Key 1 + **Secret Format**	Secreto HMAC, o privada en cualquier formato de la Parte 1
Signer 2	Segundo algoritmo y clave. Solo con `General JSON`
Claims Builder	`iss`, `sub`, `aud` y validez en horas; **Apply to Payload** los fusiona con el JSON
Payload (JSON)	Debe ser un objeto JSON de claims
Additional protected header	JSON con `kid`, `typ`, `cty`, `jku`, `x5c`, `x5t`, `x5t#S256`, `jwk`…

Reglas de la cabecera adicional: no puede fijar `alg`, `b64` ni `crit`, y una `jwk` embebida debe ser pública y asimétrica. La cabecera siempre lleva `typ: JWT` salvo que la sobrescribas.

Avisos automáticos junto al algoritmo: `none` no firma nada, y un secreto HMAC más corto que la salida del hash (32 bytes para HS256) reduce la resistencia a fuerza bruta. Con ECDSA la curva de la clave debe ser la del algoritmo.



JWT (Signed): token HS256 de tres segmentos a partir de los claims

4.2 Validate & Decode JWT

Verification Key acepta secreto, PEM, certificado, JWK o un **JWKS de varias claves**: se busca por el `kid` del token; si no hay `kid`, por `alg`; y si no, se usa la primera.

Opciones de **Advanced Validation Settings**:

Control	Efecto
Expected Issuer / Audience	Coincidencia exacta de `iss`; `aud` debe contener el valor
Clock Skew (sec)	Tolerancia de reloj. Por defecto 60
Check Expiration	Valida `exp`, `nbf` e `iat`
Strict OIDC Profile	Exige `exp`, `iat`, `iss` y `aud`
Aceptar JWT sin firma (alg=none)	Sin marcar, un token `none` nunca es válido
Confiar en la clave pública incluida en el token	Usa `jwk` o `x5c` de la cabecera cuando no das clave. Inseguro por diseño

Las comprobaciones de perfil están en el mismo formulario, dentro del desplegable **Claims Builder** del panel de generación:

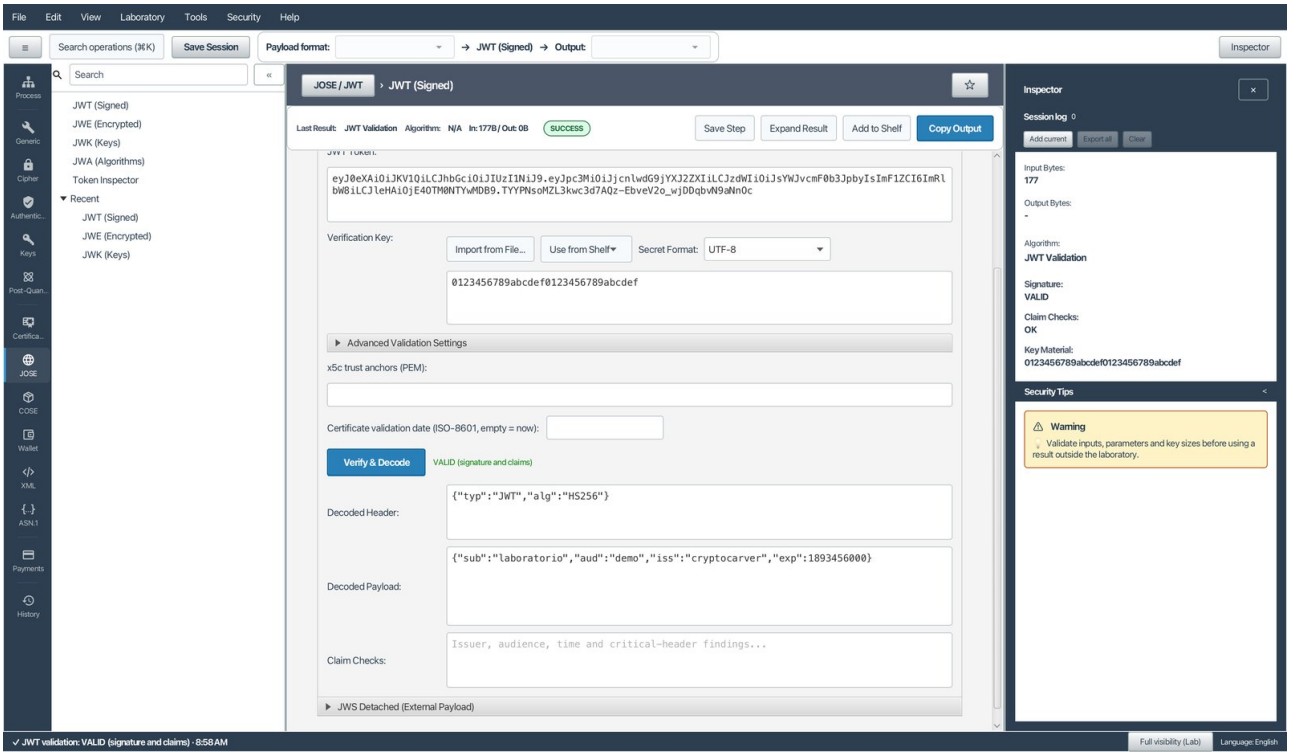
Control	Comprueba
Algoritmos JWS permitidos	Lista blanca, p. ej. `RS256, ES256`
typ esperado / cty esperado	Cabeceras `typ` y `cty`
nonce esperado	Claim `nonce`
Token de acceso para at_hash / Código de autorización para c_hash	Hashes de OIDC
cnf.jkt esperado / cnf.x5t#S256 esperado	Vinculación a clave (DPoP) o a certificado (mTLS)
Cabeceras crit conocidas	Por defecto `b64`. Un `crit` desconocido invalida el token
Ignorar crit desconocido (inseguro)	Lo degrada a aviso
Exigir perfil de token de acceso RFC 9068	Exige `iss`, `exp`, `aud`, `sub`, `client_id`, `iat`, `jti` y `typ: at+jwt`

Para tokens con `x5c`: **x5c trust anchors (PEM)** recibe las raíces de confianza y **Certificate validation date** fija la fecha de evaluación (ISO-8601; vacío = ahora). El resultado detalla cada comprobación PKIX. No hay consulta de revocación ni acceso a red.

Tres estados posibles, que conviene no confundir:

Estado	Significado
VÁLIDO (firma y claims)	Todo correcto
FIRMA VÁLIDA, CLAIMS NO VÁLIDOS	Token auténtico pero inaceptable. **Claim Checks** lista los motivos
FIRMA NO VÁLIDA	No autenticado

Usar una pública RSA como secreto de un token `HS*` se detecta como confusión de algoritmos y se rechaza con aviso.



Validate & Decode: firma y claims se comprueban por separado

4.3 JWS Detached

Firma un payload que viaja aparte. La salida compacta tiene la forma `cabecera..firma`.

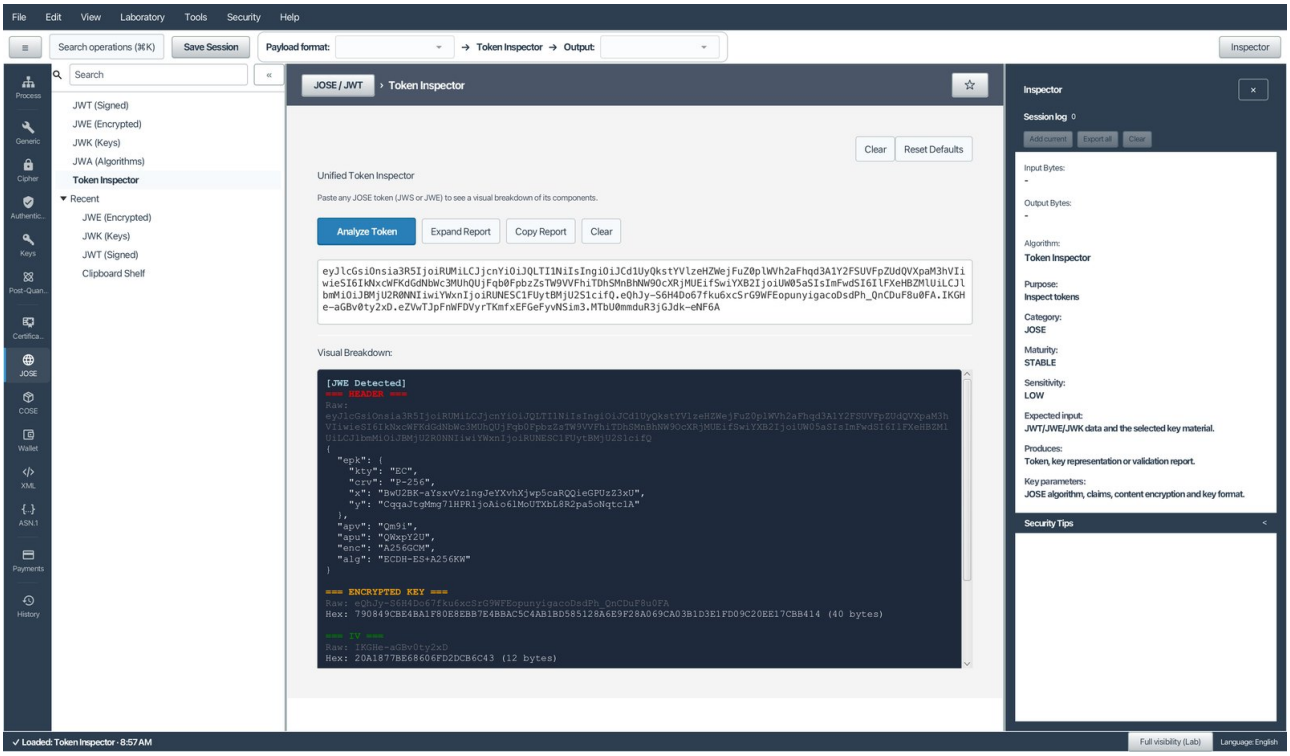
- Mismos algoritmos y serializaciones. **b64=false** firma los bytes exactos del payload externo: un salto de línea de más invalida la firma.
- **Signing Key** y **Verification Key** son campos separados; la verificación necesita el mismo **External Payload**.
- Al verificar, el algoritmo seleccionado debe coincidir con el de la cabecera. Con `General JSON` se comprueba la firma cuyo `alg` coincide.

4.4 Token Inspector

Pega cualquier token y pulsa **Analyze Token**. No pide clave y no valida nada.

Entrada	Lo que desglosa
Tres segmentos	JWS: cabecera, payload y firma
Cinco segmentos	JWE: cabecera, clave cifrada, IV, ciphertext y tag, con tamaños en bytes
JSON con `ciphertext`	JWE Flattened o General: cabecera protegida, cabecera compartida, AAD y cada destinatario
JSON con `signature(s)`	JWS Flattened o General, indicando si el payload es separado

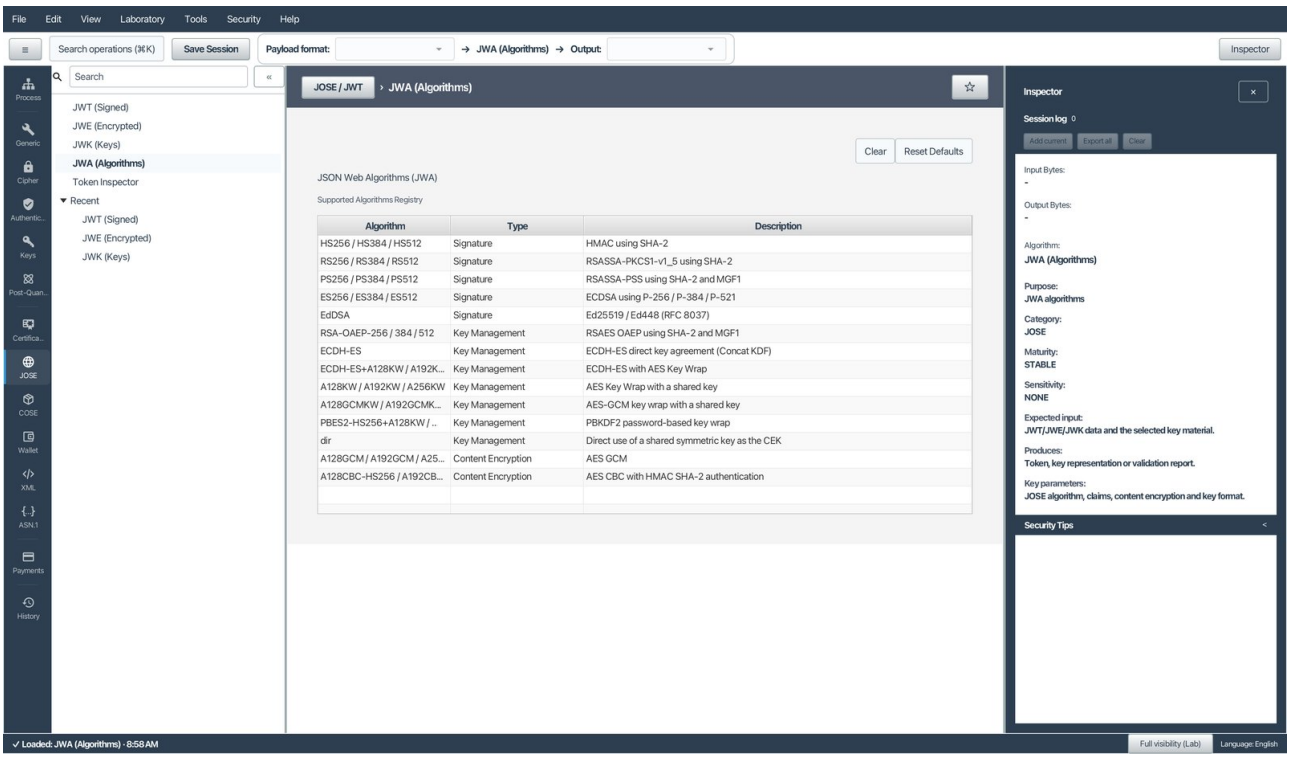
Si el payload de un JWS es a su vez un token, lo desglosa de forma recursiva. En un JWE con `cty: JWT` avisa de que dentro hay un token anidado: descífralo en **JWE** e inspecciona el resultado. **Expand Report** abre el informe en una ventana y **Copy Report** lo copia. Si la entrada contiene material privado, el informe se oculta fuera del perfil de laboratorio.



Token Inspector: desglose sin clave; se leen epk, apu y apv

4.5 JWA (Algorithms)

Tabla de consulta con los algoritmos del módulo agrupados en *Signature*, *Key Management* y *Content Encryption*. No lista los marcados como inseguros (`RSA1_5`, `RSA-OAEP`, `none`) ni `ES256K`, aunque estén disponibles en los selectores.



JWA (Algorithms): registro de algoritmos de firma, gestión de clave y contenido

4.6 JOSE en el Diseñador de procesos

Los nodos de proceso ofrecen un subconjunto más estricto:

Nodo	Alcance
`JWS_SIGN` / `JWS_VERIFY`	JWS compacto. La verificación exige que el `alg` coincida con el configurado
`JWS_DETACHED_SIGN` / `JWS_DETACHED_VERIFY`	Compacto, con opción `unencodedPayload`
`JWE_ENCRYPT` / `JWE_DECRYPT`	Solo `RSA*` y `dir`, compacto. La clave `dir` se lee como texto UTF-8
`JWT_INSPECT`	Cabecera y payload con `"verified": false`

Para ECDH-ES, AES-KW, PBES2, AAD o varios destinatarios usa la sección interactiva.

Parte 5 — Diagnóstico

Limitaciones conocidas

Área	Limitación	Alternativa
Nested	Solo `RSA*`, `ECDH-ES*` y `dir`; solo compacto	Firma en **JWT** y cifra el resultado en **JWE** → **Encrypt** con `cty = JWT`
Vista de CEK	No disponible con `dir`, `A*GCMKW`, OKP ni JSON	—
PEM cifrados	No se admiten	Descifra la clave antes

Incidencias frecuentes

Síntoma	Causa probable	Qué hacer
needs a 128-bit key (16 bytes); the supplied secret has N bytes	**Secret Format** equivocado o secreto de otro tamaño	Ajusta el formato; revisa la tabla 2.2
The … encryption method or key size is not supported con `dir`	El secreto no mide lo que exige `enc`	Tabla 2.3: CBC-HS pide el doble
The supplied key is not an RSA key	Clave EC/OKP con algoritmo RSA, o al revés	Cambia **Key Management** o la clave
The JWKS holds N keys	JWKS de varias claves donde se espera una	Pega solo la JWK; el multi-destinatario exige `General JSON`
Each recipient JWK needs an 'alg' member	JWK sin `alg` en multi-destinatario	Añade `alg` a cada clave
A symmetric algorithm needs a shared secret, not a PEM key	PEM en un algoritmo simétrico	Usa el secreto o cambia de algoritmo
JWE authentication failed…	Clave incorrecta, token alterado, AAD distinto o formato de secreto distinto	Revisa los cuatro; el mensaje no distingue a propósito
Descifra con PBES2 pero en otra herramienta no	Se pegó una JWK `oct` como contraseña	Extrae el secreto con **JWK** → **Secret**
Material de clave no válido o no compatible	**Key Type** distinto del real, `use`/`key_ops` incoherentes o `key_ops` desconocido	Revisa los tres selectores
El thumbprint no coincide con otra herramienta	Se copió la línea `// Thumbprint` o se comparan claves distintas	Compara solo la JWK; el thumbprint ignora los metadatos
La salida muestra `***MASKED***`	Perfil de visibilidad restrictivo	Cambia al perfil de laboratorio si necesitas ver la privada

Checklist antes de dar un perfil por bueno

- `alg` y `enc` fijados por política, no leídos del token sin lista blanca.
- Sin `RSA1_5`; `RSA-OAEP` (SHA-1) solo por compatibilidad.
- Tamaño exacto del secreto y **Secret Format** documentados junto al perfil.
- `kid` en cada token y una única fuente de claves configurada localmente.
- Las JWK publicadas no contienen `d`, factores RSA ni claves `oct`.
- `use` y `key_ops` coherentes con el propósito real de la clave.
- AAD acordado entre las partes si se usa serialización JSON.

- En tokens anidados: descifrar, verificar la firma y después validar claims, en ese orden.
- Round-trip completo y al menos una prueba negativa ejecutados.

Referencias

Documento	Contenido
RFC 7515	JWS
RFC 7516	JWE
RFC 7517	JWK y JWKS
RFC 7518	JWA
RFC 7519	JWT
RFC 7638	Thumbprint de JWK
RFC 7797	Payload sin codificar (`b64`)
RFC 8037	Claves OKP: Ed25519, Ed448, X25519, X448
RFC 9068	Perfil JWT para tokens de acceso